

Unidade IV

7 VERIFICAÇÃO & VALIDAÇÃO (V&V)

O objetivo da validação é demonstrar que um produto ou componente realmente atende ao propósito para o qual foi criado quando colocado em seu ambiente real de uso, ou seja, no cliente final ou em condições que o representem fielmente.

Nas metodologias ágeis, a validação ocorre de forma contínua e incremental a cada entrega, reforçando o princípio de obter feedback rápido para assegurar que o produto está alinhado às necessidades do usuário.

O principal mecanismo tradicional de validação de requisitos são as **revisões formais** apoiadas por checklists. Entretanto, em contextos ágeis, a validação é ampliada por práticas como demonstrações de sprint (sprint reviews), prototipação rápida, testes de aceitação orientados pelo cliente (acceptance test-driven development – ATDD) e definição de pronto (definition of done – DoD). A equipe envolvida deve incluir todos os interessados no software, como engenheiros de software, product owner, usuários, clientes, desenvolvedores e demais stakeholders.

A validação demonstra o quanto o produto, na sua forma atual, opera corretamente para o usuário. Para isso, utiliza atividades de verificação como testes, inspeções e simulações. No entanto, nas metodologias ágeis, essas atividades são executadas ao longo de todo o processo de desenvolvimento, e não apenas no final do projeto, garantindo que falhas sejam detectadas cedo e ajustadas rapidamente.

As atividades de verificação e validação frequentemente ocorrem em paralelo e podem compartilhar partes do mesmo ambiente, especialmente em pipelines de integração/entrega contínuas (CI/CD).

Em resumo, a **verificação** assegura que o produto foi construído corretamente com base nos requisitos do software, especificações e padrões técnicos, por meio de análises, inspeções, testes automatizados e diagnósticos. Já a **validação** confirma que o produto entregue é realmente o que o usuário precisa e que funciona adequadamente em seu ambiente operacional.

Nas abordagens ágeis, ambos os processos tornam-se contínuos, colaborativos e orientados ao valor, reduzindo riscos, aumentando a qualidade e acelerando a entrega de soluções adequadas.

7.1 Verificação do código: depuração e diagnóstico de falhas

No desenvolvimento de software, a verificação do código inclui o processo conhecido como **depuração**. No sentido clássico, depurar significa limpar, remover ou corrigir partes indesejáveis. Durante a construção do software, erros são inevitáveis, seja por travamentos, comportamentos inesperados ou falhas de lógica. Assim, a depuração consiste no rastreamento do código para identificar, corrigir e reduzir essas falhas, garantindo o funcionamento adequado do sistema.

Em termos técnicos, **depurar falhas no software (debug)** envolve acompanhar a execução do programa em tempo real, prática associada ao termo **trace (rastrear)**. O processo começa pela reprodução do problema por meio de testes, permitindo localizar o ponto exato do defeito. Uma vez encontrado, realiza-se o **diagnóstico**, que pode ser simples e imediato ou, quando necessário, formalizado em um relatório contendo a causa identificada e as soluções possíveis.

Depuração de falhas no software (debug)

A **depuração de falhas (debugging)** é a atividade responsável por rastrear e corrigir esses defeitos, garantindo que o sistema funcione conforme o esperado. Em abordagens tradicionais, a depuração costuma ocorrer após a implementação completa. Já nas metodologias ágeis, a depuração é um processo colaborativo e contínuo de entrega de valor, por incrementos, integrados ao ciclo de construção do software.

Práticas de CI, testes automatizados, TDD e programação em par (pair programming) reduzem significativamente o surgimento de defeitos e facilitam sua identificação rápida.

Para a agilidade na comunicação, são feitas reuniões curtas e frequentes, como daily meetings e daily scrum, que são reuniões diárias de acompanhamento, permitindo que a equipe compartilhe falhas encontradas, impactos de mudanças no software e tomada de decisões imediatas.

Veja a figura:

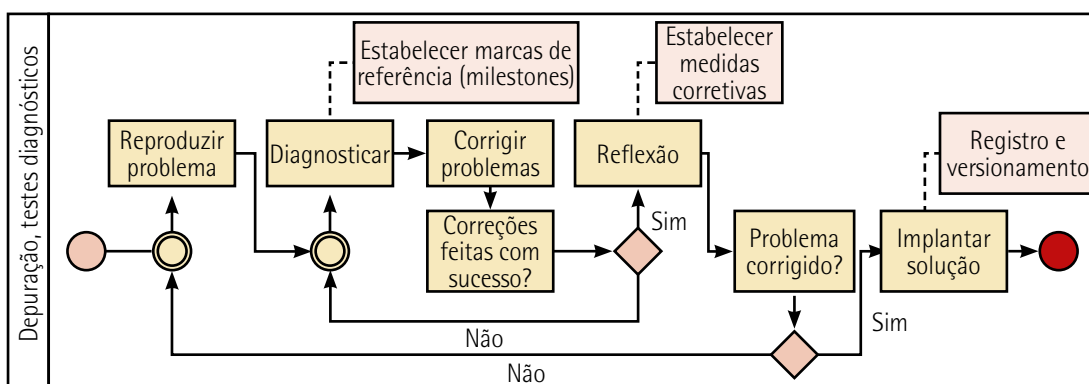


Figura 39 – Fluxo das atividades de depuração, testes e diagnósticos do software

A atividade de depuração de falhas ilustrada na figura pode ser compreendida em quatro tarefas principais:

- **Reprodução (reproduce):** consiste em rastrear o erro com o objetivo de identificar a causa do defeito. Nesse estágio, é essencial executar o programa em condições controladas e buscar reproduzir o problema por meio de breakpoints (pontos de parada no código do software) e ferramentas de inspeção para observar o comportamento do código.
- **Diagnóstico (diagnose):** avalia-se o ponto exato do erro, sua causa, gravidade, prioridade e riscos associados, bem como o impacto da falha ou mudanças que afetam o comportamento do software e em componentes que dependem ou interagem com ele. Em metodologias ágeis, é comum registrar esse diagnóstico de forma simples e objetiva, mantendo transparência para toda a equipe.
- **Correção (fix):** envolve a implementação da solução e a realização de testes para verificar se o problema foi resolvido. Em ambientes ágeis, essa etapa é fortalecida por testes automatizados, pela integração contínua e pelo TDD, que reduzem retrabalho e garantem que a correção não introduza novos defeitos.
- **Reflexão (reflect):** após a correção, medidas são aplicadas para evitar a recorrência do problema. Isso inclui verificar se a mudança não afeta outras partes do sistema, revisar entradas e saídas, otimizar o código e assegurar boas práticas como encapsulamento. No contexto ágil, a reflexão alimenta a melhoria contínua, reforçada pelas retrospectivas e aprendizado coletivo da equipe.

7.2 Testes de software: fundamentos, técnicas e práticas

O desenvolvimento de software exige técnicas de testes que garantam a qualidade, confiabilidade e evolução contínua do produto, em uma visão estruturada das principais abordagens, níveis e práticas de testes. A lógica adotada segue uma hierarquia natural no ciclo de desenvolvimento. Na sua ordem, o início se baseia em fundamentos conceituais sobre como projetar testes; em uma próxima fase, são apresentados os níveis de testes aplicados ao longo do ciclo de vida do software; e, em seguida, exploram-se práticas orientadas a testes, testes com usuários finais e métodos de garantia de qualidade.

O teste de software consiste na validação dinâmica de um sistema sob teste (SST) e fornece os comportamentos esperados em um conjunto finito de casos de teste, selecionados de forma adequada a partir do domínio de execução, que é geralmente infinito (Washizaki, 2025, p. 5).

Os testes podem ser manuais ou automatizados, envolvendo casos de teste elaborados para cobrir diversas situações. Um princípio básico na realização de testes de software, principalmente em sistemas de média complexidade para cima, é diferenciar a equipe puramente de desenvolvimento da equipe especificamente de testes, ou seja, "quem desenvolve não testa, e quem testa não desenvolve".



Saiba mais

Veja como a IBM® aborda testes de software no link a seguir:

SUSNJARA, S.; SMALLEY, I. O que são testes de software? *IBM*, [s.d.].
Disponível em: <https://tinyurl.com/4vbzzusj>. Acesso em: 30 dez. 2025.

Diversas técnicas são empregadas em diferentes momentos e de diferentes formas para validar os aspectos principais do software. Em uma ordem lógica de testes no ciclo de desenvolvimento, é recomendado que:

- 1) Inicialmente, sejam abordados os **testes caixa-preta e caixa-branca**, que formam a base metodológica para a elaboração de casos de teste, introduzindo paradigmas que orientam todas as técnicas de testes.
- 2) Na sequência e em uma ordem natural de testes, são apresentados os **testes unitário, de integração, aceitação e regressão**, da menor unidade até o produto final, que estruturam a cadeia de verificação do desenvolvimento de software, constituindo a espinha dorsal (backbone) de testes.
- 3) Na prática do desenvolvimento, organiza-se o ciclo de implementação e testes com o método ágil **TDD**, redefinindo o papel dos testes unitários ao integrá-los ao ciclo de construção do código.
- 4) Finalmente, são realizados os **testes alfa e beta**, para validar soluções em ambiente real e com usuários finais.
- 5) E na **garantia da qualidade**, usa-se uma abordagem **cleanroom**, como um método que prioriza prevenção de defeitos e testes estatísticos de confiabilidade. É um método formal que combina inspeções, desenvolvimento incremental e testes estatísticos. Dessa forma, permite-se também implementar um método complementar ou alternativo para compor o fluxo tradicional de testes.

7.3 Testes caixa-preta e caixa-branca

Os testes de software podem ser estruturados a partir de duas perspectivas da engenharia de software.

A primeira perspectiva se baseia no exame das funções especificadas para o produto. Na visão externa, avaliam-se as entradas, saídas e comportamentos observáveis do sistema, com o objetivo de confirmar que cada funcionalidade opera conforme o estabelecido e, simultaneamente, identifica discrepâncias funcionais. Essa abordagem é denominada **teste caixa-preta**, por prescindir do conhecimento da estrutura interna do software.

A segunda perspectiva fundamenta-se na análise do funcionamento interno do sistema. Na visão interna, investigam-se a arquitetura, a lógica de controle, os fluxos de execução, os módulos e os demais componentes estruturais, de modo a assegurar que todos os elementos internos foram exercitados adequadamente e que suas interações obedecem às especificações técnicas. Essa estratégia caracteriza o **teste caixa-branca** e se apoia em informações derivadas do código-fonte, assim como em métricas estruturais de software, tais como cobertura de instruções, de decisões e de caminhos.

As duas abordagens são complementares, uma vez que a caixa-preta enfatiza a conformidade funcional e comportamental, a caixa-branca evidencia a qualidade estrutural, o desempenho interno e aspectos de segurança associados à implementação.

No contexto de sistemas desenvolvidos sob encomenda ou configurados para atender necessidades específicas de um domínio de negócio, ambas as perspectivas subsidiam o processo de **verificação e validação (V&V)**. Dessa forma, os testes caixa-preta e caixa-branca, aplicados de maneira sistemática e integrada, contribuem para a conformidade técnica, robustez estrutural e adequação do software ao propósito para o qual foi concebido.

Conforme ilustrado na figura a seguir, o teste caixa-preta, também denominado teste comportamental, é uma técnica de avaliação na qual o software é analisado exclusivamente a partir de suas interfaces externas, sob a ótica do usuário, sem que o testador tenha acesso ou necessidade de compreender sua estrutura interna, lógica de implementação ou arquitetura.

O foco dessa abordagem é verificar se o sistema atende aos RF e RNF especificados, validando o comportamento observado mediante diferentes combinações de entradas e condições de uso. Dessa forma, o teste concentra-se na conformidade do software com o que foi solicitado, independentemente de como a solução foi construída internamente.

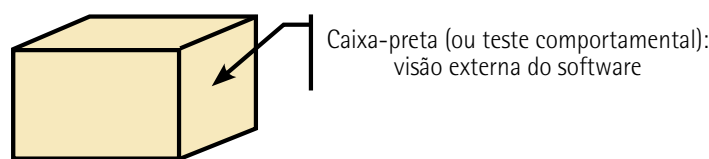


Figura 40 – Representação de caixa-preta em que o testador não consegue ver seu interior

Os testes caixa-preta concentram-se na verificação do comportamento do software a partir da relação entre os requisitos do usuário (RU), RNF e RF, avaliando exclusivamente se as entradas fornecidas e as saídas obtidas correspondem ao que o usuário final espera.

Essa abordagem é utilizada em métodos ágeis, nos quais a validação contínua do valor entregue ao usuário, por meio de user stories, trata de critérios de aceitação. Testes orientados ao comportamento (BDD) constituem uma base do processo de testes. Eles não analisam a lógica interna do código, mas sim o quanto a funcionalidade entregue atende ao que foi acordado com o cliente, a cada incremento.

Exemplo de aplicação

Estudo de caso: teste caixa-preta para aplicativo de e-commerce

Em um aplicativo de e-commerce, a equipe realiza testes caixa-preta para avaliar se as funcionalidades descritas nas user stories contemplam funcionalidades como adicionar itens ao carrinho, finalizar a compra e efetuar o pagamento. Se elas estão claras para o usuário, atendem aos critérios de aceitação e operam corretamente.

Observe que toda a validação ocorre sem inspeção do código-fonte, com foco exclusivo no comportamento observado do sistema e na experiência do usuário.

Para a execução tanto de testes caixa-preta quanto caixa-branca, recomenda-se, inicialmente, a elaboração de casos de teste estruturados, que podem ser organizados em um quadro padronizado, como o exemplo apresentado a seguir, voltado para testes caixa-preta.

No quadro a seguir, a primeira coluna apresenta os códigos identificadores dos casos de teste (CT), a segunda coluna registra os requisitos associados a cada teste, no caso requisito funcional (RF) e a terceira e última coluna descreve a especificação detalhada do teste, incluindo condições de entrada, comportamento esperado e resultados previstos.

Quadro 19 – Casos de testes caixa-preta com base nos RF

Caso de teste	Ref.: lista de requisitos	Especificação
CTCXP 01	RF01	CTCXP 01: Gestão de Usuários (cadastro e autenticação) 1) Verificar se o menu de Gestão de Usuários apresenta as opções corretas conforme requisitos 2) Validar o cadastro, confirmando armazenamento dos dados e mensagens esperadas 3) Conferir se textos e mensagens são exibidos de forma completa e clara 4) Revisar o processo de autenticação, incluindo regras de validação e segurança

CTCXP – Caso de teste caixa-preta

RF – Requisito funcional

Observe a figura 41 a seguir. O teste caixa-branca, também chamado teste da caixa-de-vidro ou teste estrutural (Pressman; Maxim, 2021, p. 19), consiste em uma abordagem de verificação orientada à análise da estrutura interna do software. Nessa técnica, o testador possui acesso direto ao código-fonte e ao fluxo de controle do programa, permitindo identificar erros relacionados à lógica interna, ao uso inadequado de variáveis, à cobertura insuficiente de caminhos e ao comportamento estrutural dos componentes.

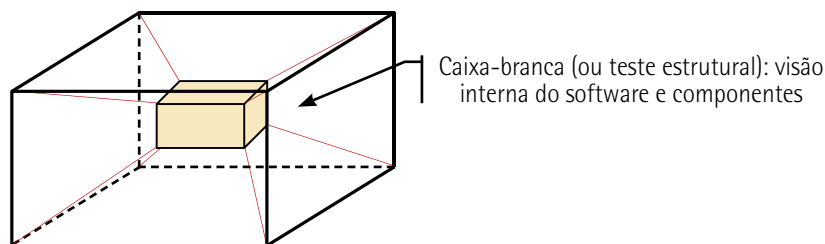


Figura 41 – Representação uma caixa-branca em que o testador consegue ver seu interior

O teste caixa-branca caracteriza-se por casos de testes baseados na estrutura de controle definida no nível de componentes. Assim, os casos de teste são derivados a partir de instruções, decisões, nós, blocos lógicos e demais elementos internos da implementação, garantindo que todas as rotas críticas, decisões condicionais e estruturas de repetição sejam exercitadas de forma abrangente.

Nas metodologias ágeis, especialmente em práticas como XP e TDD, os desenvolvedores escrevem testes estruturais antes da implementação do código, o que favorece alta cobertura, ciclos curtos de feedback e detecção precoce de falhas estruturais. O teste caixa-branca não apenas apoia a atividade de verificação, como também fortalece a testabilidade, a qualidade interna e a manutenibilidade.

Os testes caixa-branca são fundamentados na relação direta entre os RF e os requisitos de sistema (RS), concentrando-se na análise do código-fonte, dos mecanismos de acesso à estrutura do banco de dados e das interfaces de hardware e rede de computadores, que suportam o funcionamento do backend. Essa abordagem permite verificar se todas as estruturas internas do sistema, como rotinas de cálculo, validações, fluxos condicionais, chamadas de serviços, transações de dados e integração, estão implementadas conforme especificado.

Exemplo de aplicação

Caso: teste caixa-branca aplicado em sistema de controle de estoque

Em um sistema de controle de estoque, os testadores executam testes caixa-branca para avaliar se todas as condições lógicas relacionadas aos limites de estoque – mínimo, máximo, reposição automática com base no algoritmo de lote econômico de compra (LEC) e bloqueios de saída – estão sendo percorridas, calculadas e tratadas de forma correta.

Mesmo que a lógica de cálculo esteja correta, pode haver problemas de acesso aos dados, devido a problemas na rede ou até mesmo em seu servidor, omitindo dados ou tratando nos cálculos os dados corrompidos. Dessa forma, o backend pode apresentar falhas em camadas de acesso a dados, como consultas incompletas, registros não retornados, transações parcialmente concluídas ou inconsistências na comunicação com o banco de dados, sendo que qualquer falha desse tipo pode resultar em valores omitidos ou incorretos, comprometendo a precisão do cálculo e, consequentemente, gerando erros que afetam diretamente o comportamento do sistema.

No quadro a seguir, a primeira coluna apresenta os códigos identificadores dos casos de teste (CT), a segunda coluna registra os requisitos associados a cada um deles, no caso requisito do sistema (RS), e a terceira e última coluna descreve a especificação detalhada do teste, incluindo condições de entrada, comportamento esperado e resultados previstos.

Quadro 20 – Casos de testes caixa-branca que associam o caso de teste com os requisitos do sistema

Caso de teste	Ref.: Lista de requisitos	Especificação
CTCXB 01	RS01	CTCXB 01: Processamento interno – regras de controle de estoque 1) Avaliar o fluxo lógico interno que calcula estoque mínimo, máximo e pontos de reposição 2) Verificar instruções condicionais, laços e estruturas de decisão relacionadas ao controle de saldo 3) Testar o comportamento do backend diante de falhas simuladas no banco de dados (exemplo: registros ausentes ou desatualizados) 4) Confirmar integridade das transações e consistência dos valores armazenados
CTCXB 02	RS03	CTCXB 02: Acesso a dados – camada de persistência 1) Validar a execução das queries e stored procedures utilizadas na atualização de estoques 2) Verificar tratamento de exceções internas na persistência, como timeouts, deadlocks e conexões inválidas 3) Avaliar se todas as rotas lógicas de consulta são percorridas, incluindo caminhos de erro 4) Testar consistência das transações ACID (atomicidade, consistência, isolamento e durabilidade)
CTCXB 03	RS05	CTCXB 03: Integração interna – serviço de mensagens 1) Analisar chamadas internas entre módulos do backend responsáveis pela entrada e saída de produtos 2) Avaliar a lógica de validação de dados recebidos de APIs internas 3) Verificar tratamento de inconsistências entre camadas de serviço e domínio 4) Confirmar a correta transferência de exceções e mensagens internas

CTCXB – Caso de teste caixa-branca

RS – Requisito do sistema

7.4 Testes unitário, de integração, de aceitação e de regressão

No desenvolvimento ágil, a qualidade é construída desde o início e mantida continuamente ao longo de todo o ciclo de desenvolvimento. Como as entregas ocorrem de forma incremental e frequente, cada parte do software é validada de maneira rápida, objetiva e confiável.

O atributo da qualidade **testabilidade** refere-se à facilidade com que um software pode ser testado, como a clareza de seu design, da modularidade do código, da existência de interfaces bem definidas ou do suporte à automação. Quanto maior a testabilidade, mais eficiente e confiável se torna a execução dos diferentes tipos de testes.

As práticas ágeis fortalecem esse atributo ao adotar princípios como simplicidade, design limpo, integração contínua e ciclos curtos de feedback. Dessa forma, os testes unitário, de integração, de aceitação e de regressão não apenas verificam o funcionamento do software, como também beneficiam-se por um índice maior de testabilidade, resultando em um desenvolvimento orientado à qualidade.

7.4.1 Teste unitário (unit test)

O objetivo do **teste unitário** é testar componentes pequenos e isolados do software, como funções, métodos ou classes, de forma a assegurar que cada unidade se comporte conforme o esperado. Esses testes são executados continuamente pelo próprio desenvolvedor durante a codificação, permitindo identificar erros rapidamente e evitando que defeitos avancem para fases posteriores.

No desenvolvimento ágil, práticas como TDD e integração contínua, constituem uma base da garantia de qualidade. Os testes unitários compõem parte essencial do definido como pronto (DoD) e alimentam o ciclo de integração/entrega contínua, com feedbacks rápidos sobre a estabilidade e comportamento do código.

Em conformidade com um plano de garantia da qualidade (GQS), a ISO/IEC 25010 define o modelo de qualidade de produto e o modelo de qualidade em uso, estabelecendo características e subcaracterísticas que devem ser avaliadas no software. As relações da ISO/IEC 25010 existentes com teste unitário têm o foco em funções isoladas, verificando se cada unidade está correta e contribuindo para avaliar:

- **funcionalidade/adequação funcional:** confere se cada função implementada corresponde ao esperado;
- **confiabilidade/maturidade:** identifica defeitos antes que ocorram, reduzindo risco de falhas posteriores;
- **manutenibilidade/modularidade:** quanto mais modular o código, mais testável ele é (subcaracterística testabilidade);
- **desempenho (em pequena escala):** avaliações iniciais de eficiência de métodos específicos.

No que se refere ao emprego de ferramenta ágil, em um sistema de gerenciamento de transportes, o desenvolvedor cria testes unitários para validar a função de roteirização antes mesmo de implementar a lógica final, seguindo TDD. Com o método caixa-branca, ele verifica se cada regra de cálculo segue o comportamento esperado descrito na história de usuário.

As ferramentas mais apropriadas a serem utilizadas são as que suportam a criação e execução de testes unitários e o cálculo de métricas de cobertura de código. Dessas, destacam-se:

- **Frameworks de teste unitário:** são ferramentas que permitem que o desenvolvedor escreva e execute o código de teste antes do código de produção, conforme o TDD.
- **Ferramentas de cobertura de código (code coverage):** o método caixa-branca (ou white-box testing) tem o foco na estrutura interna do código. O desenvolvedor precisa saber se o código de teste está realmente executando (e, portanto, verificando) cada linha e cada regra de cálculo.

- **Frameworks para mocks/stubs (isolamento de teste):** em um sistema de roteirização, a função pode depender de serviços externos (como um API de mapas) ou componentes complexos do sistema (como um banco de dados). Para garantir que o teste unitário seja um teste de caixa-branca puro, ou seja, apenas um teste com foco na lógica da função, é necessário identificá-lo e isolá-lo como um componente, por exemplo.

7.4.2 Teste de integração (integration test)

Após os testes unitários, as unidades são integradas para formar novos componentes ou módulos. Na abordagem ágil, os **testes de integração** ocorrem de forma incremental, acompanhando as entregas de cada sprint. Seu objetivo é verificar a comunicação e a interação entre módulos, APIs, serviços e componentes do sistema, garantindo que o conjunto funcione de maneira coesa. Esses testes fazem parte do ciclo automatizado que ajudam a detectar falhas de integração assim que surgem, evitando acúmulo de erros e retrabalho no final da iteração.

Em conformidade com a ISO/IEC 25010, o teste de integração verifica se os componentes funcionam corretamente juntos, contribuindo para avaliar:

- **compatibilidade/interoperabilidade:** a integração só funciona quando módulos intercambiam dados corretamente;
- **confiabilidade/maturidade e tolerância a falhas:** garante funcionamento estável entre componentes;
- **funcionalidade/completude funcional:** valida funções que dependem de múltiplas partes do sistema;
- **segurança/controle de acesso entre módulos:** aplicada em cenários específicos.

Exemplo de aplicação

Estudo de caso: teste de integração – arquitetura de sistema de emissão de nota fiscal

Durante a sprint, o time integra o componente Emissão de Nota Fiscal com os serviços externos do cliente. No ambiente de integração (parte do CI), são testados a troca de dados, chamadas de API, comunicação com dispositivos e compatibilidade com o ambiente operacional. Apenas após essa validação, o item pode ser movido para a próxima etapa, que é pronto para aceitação (ready for acceptance).

Aqui, os testes de integração iniciam-se pelos testes do projeto da arquitetura. Nesse caso, a modelagem das arquiteturas de infraestrutura se faz com o uso de diagramas da UML, como diagrama de casos de uso, de estrutura, de sequência e de implantação. Podem ser usadas também ferramentas como Astah (gratuita e corporativa) ou Draw.io (gratuita).

7.4.3 Teste de aceitação (acceptance test)

O **teste de aceitação**, ou teste de validação, ocorre quando o incremento já passou pelos testes unitários e de integração. No desenvolvimento ágil, ele está diretamente ligado às histórias do usuário, critérios de aceitação e testes orientados ao comportamento/aceitação (behavior-driven development/ acceptance test-driven development – BDD/ATDD).

Seu objetivo é confirmar que o software atende às necessidades do cliente e entrega o valor descrito nos RF e RNF. Estando na sprint review ou em ciclos de validação contínua, o cliente ou product owner realiza o aceite do incremento entregue.

Em conformidade com a ISO/IEC 25010, o teste de aceitação valida se o sistema atende aos requisitos do cliente e do usuário final, contribuindo para avaliar:

- **qualidade em uso/satisfação do usuário:** testes de aceitação confirmam que o sistema entrega valor ao usuário;
- **funcionalidade/adequação funcional:** se o software atende às necessidades previstas;
- **usabilidade/adequação ao uso:** avalia experiência, mensagens, fluxos e clareza;
- **segurança/integridade e confidencialidade:** em sistemas sensíveis, o aceite inclui validação de política de segurança.

Exemplo de aplicação

Estudo de caso: testes de aceitação no e-commerce B2C

Em um ambiente de e-commerce B2C, o time executa testes de aceitação para validar todo o fluxo de compra descrito nas histórias de usuário: adicionar produtos ao carrinho, calcular o frete, efetuar o pagamento e registrar o pedido. O aceite do cliente confirma que o incremento está pronto para produção.

O desenvolvimento ágil dos testes de aceitação pode ter início com a modelagem ágil (AM), que é um conjunto de práticas e princípios para modelar e documentar sistemas de software de forma eficaz e leve, servindo de apoio a outras metodologias ágeis.

O quadro a seguir (quadro 21) apresenta um conjunto de princípios ágeis e evidencia os principais eventos do ciclo de desenvolvimento nos quais a modelagem ágil é aplicada. O objetivo é demonstrar como práticas de modelagem leve, iterativa e colaborativa apoiam a compreensão do domínio, a definição de requisitos e a tomada de decisão contínua ao longo das iterações.

Quadro 21 – Casos de testes caixa-branca que associam o caso de teste com os requisitos do sistema

Evento/momento	Prática de modelagem ágil aplicada
Planejamento da sprint	Modelagem colaborativa/esboços: aqui o objetivo, bem específico, é transformar o requisito em um modelo de comportamento testável. Use um esboço rápido para garantir que todos entendam as interfaces e dependências das histórias de usuário mais complexas
Desenvolvimento (coding)	Modelagem JIT/evolução contínua: o teste de aceitação é aplicado de forma prática através da sua automação e da adoção do TDD, garantindo que o código construído atenda exatamente ao modelo de comportamento definido. Antes dessa atividade, desenhe rapidamente um diagrama de classe no seu IDE ou um diagrama de componentes em até mesmo um papel antes de uma refatoração crucial
Integração/testes (CI)	Modelos de trabalho: o estágio de integração e testes contínuos é onde o teste de aceitação tem sua maior importância, por automatizar o ciclo de desenvolvimento ágil, transformando-se em um modelo de validação da arquitetura do sistema. A cobertura de código se torna os modelos de comportamento e qualidade do sistema
Revisão da sprint (review)	Modelar com propósito: serve de critério final de verificação para que o product owner e stakeholders validem se o incremento de software atende às expectativas e aos requisitos de negócio. Use um diagrama simples (como um diagrama de implantação) para explicar como a nova funcionalidade foi integrada ao ambiente

7.4.4 Teste de regressão (regression test)

No desenvolvimento ágil, em que mudanças são frequentes e incrementos são liberados rapidamente, o **teste de regressão** garante que mudanças no código, como correções, melhorias ou novas funcionalidades, não introduzam erros em partes já estáveis do sistema. Em ambientes ágeis maduros, os testes de regressão são amplamente automatizados, permitindo verificar rapidamente todo o sistema a cada build gerada.

Em conformidade com a ISO/IEC 25010, o teste de regressão garante que mudanças não impactam em funcionalidades já existentes, contribuindo para avaliar:

- **Confiabilidade/estabilidade:** confere que o sistema continua funcionando ao longo das modificações.
- **Manutenibilidade/modificação e testabilidade:** quanto mais fácil adaptar o sistema sem gerar falhas, maior a manutenibilidade.
- **Desempenho (continuidade):** as mudanças não devem degradar a performance geral.
- **Segurança:** alterações não podem introduzir vulnerabilidades.

Em um módulo financeiro de um ERP, um defeito em um cálculo é corrigido durante a sprint. Após o ajuste, os testes de regressão automatizados são executados para garantir que o conserto não afetou outras rotinas como cálculos de juros, fechamento mensal ou exportação contábil. Quando necessário, o desenvolvedor aplica depuração reversa para localizar a origem da inconsistência e garantir que nenhuma outra funcionalidade foi comprometida.

A técnica de depuração reversa inverte o fluxo tradicional de análise. Ela ocorre no sentido inverso de rastreamento, ou seja, parte da informação, que é o resultado final, e caminha para os dados que a geraram, resultantes de algoritmos e interfaces usadas na construção da informação. Essa é uma abordagem top-down que verifica a integração dos componentes de um sistema, começando pelos níveis superiores, principalmente as que fazem interface com o operador do software, e caminha para os níveis inferiores, incluindo o projeto e testes do código.

7.5 Design e comportamento: FDD, DDD e abordagens orientadas a testes

As práticas de **design e comportamento** representam uma sinergia essencial para mitigar riscos e otimizar a construção de software. A análise e intersecção de metodologias como o Feature-Driven Development (FDD), ou Desenvolvimento Dirigido por Funcionalidade, que estrutura o processo em torno de entregas de valor, e o Domain-Driven Design (DDD), ou Design Orientado a Domínio, que fornece os fundamentos arquiteturais para modelar domínios complexos, permitindo a criação de sistema de software robusto e alinhado à necessidade do negócio, capaz de suportar arquiteturas com fluxos de trabalho ágeis. Juntas, essas abordagens criam uma estrutura coesa para o projeto, garantindo que o escopo (FDD) e o modelo interno (DDD) estejam alinhados às necessidades do negócio.

A complementação dessa estrutura ocorre por meio das **abordagens orientadas a testes**, que transformam requisitos em especificações executáveis, garantindo a qualidade em diferentes níveis. Enquanto o Acceptance Test-Driven Development (ATDD), ou Desenvolvimento Guiado por Testes de Aceitação, e o Behavior-Driven Development (BDD), ou Desenvolvimento Guiado Pelo Comportamento, focam na colaboração para definir e automatizar o comportamento esperado do sistema a partir da perspectiva do usuário. O Test-Driven Development (TDD), ou Desenvolvimento Guiado por Testes, impulsiona o design do código-fonte, assegurando a confiabilidade e a manutenibilidade das unidades de software.

A figura 42, a seguir, mostra os pilares da abordagem ágil, mencionados no design e comportamento do software.

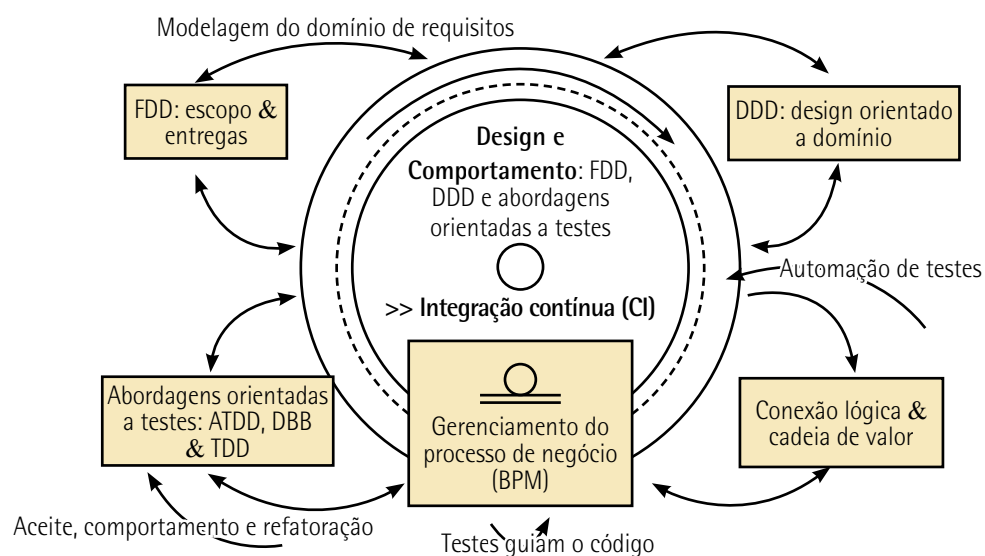


Figura 42 – Design e comportamento: FDD, DDD e abordagens orientadas a testes

A aplicação integrada dos pilares da abordagem ágil é fundamental para qualquer engenheiro de software buscar não apenas construir o sistema rapidamente, mas garantir que ele seja correto, sustentável e fiel ao domínio de negócio.

- **Fundamentos e metodologias ágeis de alto nível:** são as estruturas de desenvolvimento que definem o fluxo de trabalho e o ciclo de vida da entrega. O **FDD** é uma metodologia ágil que organiza o desenvolvimento e o planejamento em torno das funcionalidades (features) do negócio. O FDD utiliza práticas de teste e design com base no TDD/BDD para garantir que cada feature entregue seja de alta qualidade e funcione conforme o esperado pelo cliente.
- **Foco na arquitetura e no domínio:** o foco está na estrutura do software para lidar com a complexidade do negócio (domínios). O DDD também se beneficia do TDD e BDD para garantir que o modelo do domínio de negócio (entidades, interfaces e controle) funcione exatamente como o negócio espera, usando os testes como uma forma de documentar e validar o comportamento.
- **Práticas orientadas a testes e requisitos (test-driven):** detalha as práticas que garantem que o time construa o que é certo de maneira correta.
 - **Base da qualidade do código:** o **TDD** é a prática de escrever testes unitários antes do código de produção para impulsionar o design e garantir a qualidade interna do código. É o "como" construir bem.
 - **Conexão com o negócio:** o **ATDD** é uma filosofia com foco na colaboração para definir os testes de aceitação, que são os critérios de "pronto" do cliente, antes da codificação, de forma a garantir que o time construa a funcionalidade correta. Além disso, garante que está sendo construída a coisa certa, definindo "o que" antes de começar.
 - **Ferramenta/prática:** o **BDD** é a técnica que implementa o ATDD, usando a sintaxe Gherkin (Dado/Quando/Então) para expressar testes de aceitação em uma linguagem que todos envolvidos no projeto (técnicos e não técnicos) possam entender.
- **Medição e execução de processos:** é a gestão do trabalho e acompanhamento do desempenho da equipe.
 - **Business Process Management (BPM), ou Gerenciamento de Processos de Negócio:** é uma disciplina que introduz a modelagem do processo de negócio como base para gerenciar, analisar, automatizar e melhorar os processos de negócio.
- **Práticas de entrega contínua:** é a prática automatizada do fluxo de trabalho. Aborda:
 - **CI/commit no pipeline (envio de mudanças ao repositório):** o CI é a prática de integrar o código frequentemente e, em seguida, o commit descreve o envio de mudanças ao repositório como o evento que dispara toda a sequência de validação e testes automatizados.

7.6 Testes alfa e beta

Como vimos anteriormente, a validação contínua do software busca garantir que o produto atenda às necessidades reais do usuário e evolua de acordo com feedbacks. Os **testes alfa e beta** são práticas que ampliam a visão sobre a qualidade, usabilidade e adequação do software em diferentes ambientes e perfis de uso.

Quando o software é desenvolvido para apenas um cliente, o teste de aceitação conduzido pelo próprio usuário costuma ser suficiente. Porém, em produtos voltados para múltiplos clientes ou mercados distintos, o teste de aceitação isolado não cobre as variáveis de contexto, diferenças culturais, particularidades regionais ou variações de infraestrutura tecnológica. Dessa forma, a estratégia mais eficaz é incorporar testes alfa e beta como parte do ciclo iterativo de desenvolvimento, permitindo feedbacks estruturados, ajustes contínuos e validação em múltiplos ambientes.

O **teste alfa** ocorre em um ambiente controlado, geralmente dentro da própria empresa desenvolvedora. A equipe de desenvolvimento, a Quality Assurance (QA), ou Garantia da Qualidade, e usuários internos (cliente e desenvolvedores) simulam diferentes cenários de uso desde as primeiras etapas do produto, incluindo instalação, configuração, interação e operação.

Essa etapa funciona como um feedback inicial da qualidade, permitindo identificar falhas, inconsistências, problemas de usabilidade e pontos de melhoria antes de disponibilizar o software para usuários externos. Além disso, práticas como testes exploratórios, sessões de pair testing, registros de incidentes e reuniões de revisão ajudam a gerar insights rápidos e acionáveis.



Observação

Uma empresa de web design reúne desenvolvedores, designers e alguns funcionários para testar internamente uma aplicação web. Em um ambiente interno e preparado, são registradas reações, comentários, dificuldades de uso e sugestões. Esses dados alimentam o backlog e impulsionam melhorias nas próximas iterações.

O **teste beta** é a validação em ambiente real com feedbacks de usuários finais. Após os ajustes resultantes do teste alfa, o software é liberado para o **teste beta**, que é realizado no ambiente real do usuário, ou seja, em ambiente descontrolado e sem acesso direto da equipe de desenvolvimento. Esse teste forma a base para observações do comportamento do software diante de diferentes configurações de hardware, sistemas operacionais, redes e domínios de uso.

Nas metodologias ágeis, o teste beta possibilita feedbacks diretos do usuário, em apoio a decisões de refinamento nas próximas sprints. Ferramentas como coleta automática de métricas, sistemas de feedback integrado, FAQ dinâmico, tracking de erros e monitoramento de comportamento do usuário (telemetria) ajudam a equipe a compreender a experiência real e priorizar melhorias nas sprints.



Observação

A mesma empresa de web design libera a aplicação para um grupo diversificado de usuários, permitindo o uso em diferentes sistemas e dispositivos. Os feedbacks e relatórios automáticos (como erros, travamentos, percepções de usabilidade, sugestões e FAQs) ajudam a equipe a avaliar a experiência real e identificar ajustes necessários.

Versões beta de software

A instalação e utilização de uma versão beta de um produto de software deve ser precedida por uma avaliação consciente do risco, porque tais versões, por natureza, representam estágios preliminares do ciclo de vida do desenvolvimento, sendo disponibilizadas para testes e validação em ambientes reais.

Recomenda-se que o usuário pondere a falta de proficiência técnica para mitigar falhas. Os riscos potenciais incluem, mas não se limitam a:

- **instabilidade operacional:** frequentes falhas de sistema, comportamentos inesperados e queda de desempenho;
- **incompatibilidade no ambiente operacional:** problemas de integração e funcionalidade adversa com configurações de hardware e versões do sistema operacionais não validadas;
- **vulnerabilidades de segurança:** presença de bugs de segurança não corrigidos que podem expor o sistema a ameaças cibernéticas;
- **limitações de suporte:** ausência ou redução de suporte técnico, exigindo do usuário a capacidade de diagnosticar e resolver problemas por conta própria;
- **consumo excessivo de recursos:** subutilização de memória, processamento e armazenamento, impactando a eficiência global do sistema computacional.



Lembrete

A instalação e utilização de uma versão beta de um produto de software deve ser precedida por uma avaliação consciente do risco.

8 GESTÃO DE MUDANÇAS E EVOLUÇÃO DO SOFTWARE

A **gestão de mudanças e a evolução do software** constituem elementos estruturais no ciclo de vida de sistemas de software, em contextos organizacionais dinâmicos e altamente dependentes de tecnologias digitais. À medida que requisitos de negócio, regulamentações, tecnologias emergentes e expectativas dos usuários se transformam, os sistemas precisam ser continuamente adaptados para garantir funcionalidade, confiabilidade e aderência ao propósito para o qual foram concebidos.

Esse processo não se limita a simples correções de erros, mas envolve um conjunto estruturado de práticas que asseguram a análise, o controle e a implementação de modificações de forma planejada, minimizando riscos e preservando a integridade do produto. Assim, compreender os mecanismos de gestão de mudanças torna-se indispensável para equipes de desenvolvimento que atuam em ambientes de software complexos e em constante evolução.

A evolução do software, por sua vez, representa um fenômeno natural e inevitável, decorrente do uso contínuo e da necessidade de adaptação ao longo do tempo. Estudos clássicos, como as Leis de Lehman (Pressman; Maxim, 2021), demonstram que sistemas de software tendem a crescer em complexidade e requerem manutenção contínua para permanecerem úteis e competitivos.

Nesse cenário, metodologias ágeis, práticas DevOps e processos de engenharia avançada desempenham um papel fundamental ao integrar ciclos curtos de feedback, automação e estratégias sistemáticas de versionamento e configuração.

Ao abordar a gestão de mudanças de maneira estruturada e alinhada à evolução contínua do software, as organizações conseguem responder rapidamente às demandas do mercado, promover inovação sustentável e garantir a qualidade do produto final.

8.1 Fundamentos da gestão de mudanças

Entre as particularidades que distinguem a engenharia de software das demais áreas da engenharia, destaca-se o caráter contínuo de manutenção e evolução dos sistemas. Diferentemente de produtos físicos, cuja expectativa é chegar ao usuário totalmente pronto e funcional, o software passa por ajustes desde sua implantação, seja para resolver problemas de configuração, adequações ao ambiente operacional, incompatibilidades de interface ou atender novos requisitos. Isso ocorre porque o software é um artefato vivo: usuários descobrem novas necessidades, identificam oportunidades de melhoria e demandam maior usabilidade, funcionalidade e integração, que são fatores que impulsionam mudanças contínuas no software.

A manutenção de software, portanto, constitui o conjunto estruturado de modificações realizadas após sua liberação, englobando correções, melhorias, adaptações e evolução funcional. Pressman e Maxim (2021) destacam o processo de manutenção relevante e estratégico em sistemas customizados, nos quais diferentes equipes atuam antes e depois da implantação do sistema de software, exigindo mecanismos rigorosos de gestão de configuração, testes contínuos, versionamento e comunicação eficaz.

Essa dinâmica se intensifica com equipes trabalhando em ciclos curtos, priorizando feedback rápido, automação de testes e integração contínua para manter a qualidade, reduzir riscos e garantir que o software acompanhe, de forma responsiva, as necessidades em constante transformação dos usuários e do negócio.

Para fornecer suporte eficaz ao software de classe industrial, a empresa (ou seus designados) deve ser capaz de fazer correções, adaptações e melhorias inerentes à atividade de manutenção. Além disso, a empresa deve desempenhar outras atividades importantes, que incluem suporte operacional continuado, suporte ao usuário e atividades de reengenharia durante toda a vida útil do software (Pressman; Maxim, 2021, p. 1121).

A manutenção pode assumir **caráter corretivo, preventivo** ou **adaptativo**, permitindo ajustes contínuos conforme evoluem as tecnologias, os processos organizacionais e as demandas dos usuários. Essa manutenção é potencializada por práticas como feedback rápido, priorização dinâmica no backlog, integração e entregas contínuas (CI/CD), e colaboração ativa entre desenvolvedores, testadores e stakeholders. Com essas habilidades e ciclos iterativos, as organizações asseguram a qualidade, a segurança e a usabilidade do software, promovendo um ambiente de TI mais confiável, responsivo e alinhado às necessidades do negócio.

A manutenção como estratégia sustenta atributos de qualidade como confiabilidade, segurança, usabilidade, manutenibilidade e desempenho, garantindo a qualidade do software e promovendo um ambiente de TI confiável e eficiente.

A manutenção corretiva, preventiva ou adaptativa permite ajustes conforme as mudanças tecnológicas, correções e demandas de negócios e dos usuários. Por meio da manutenção de software, as organizações asseguram a qualidade, segurança e usabilidade de suas aplicações.

Vejamos cada uma delas em detalhes.

Manutenção corretiva

Tem como objetivo restaurar o software ao seu comportamento funcional esperado, garantindo que atenda aos requisitos e expectativas dos usuários. Grande parte dessa atividade corresponde ao processo de depuração de falhas (debugging, comentado no tópico 7.1 Verificação do código: depuração e diagnóstico de falhas), que envolve as tarefas de rastreamento do erro, diagnóstico, correção e reflexão.

Para aprimorar a análise de defeitos em uma manutenção corretiva, é importante integrar o uso de ferramentas de debug com técnicas estruturadas de **análise de causa raiz** (do inglês root cause analysis – RCA). A técnica sugerida é o diagrama de causa e efeito (diagrama de ishikawa ou diagrama de espinha de peixe), orientado pelo PMBOK® (2017), como mostra a figura a seguir, que analisa o sistema de software GUESS.

O **diagrama de causa/efeito** tem por objetivo visualizar, categorizar e mapear as possíveis causas que contribuem para um efeito específico (ou problema) que se deseja analisar.

No diagrama apresentado na figura a seguir, observe o bloco principal (cabeça do peixe) à direita, com o nome do "efeito" em análise, que é "GUESS – SISTEMA DE SOFTWARE", as hastes (espinhas), com as principais categorias da "causa" (os componentes de infraestrutura), e em cada ramificação menor, as possíveis causas que podem levar a esse efeito.

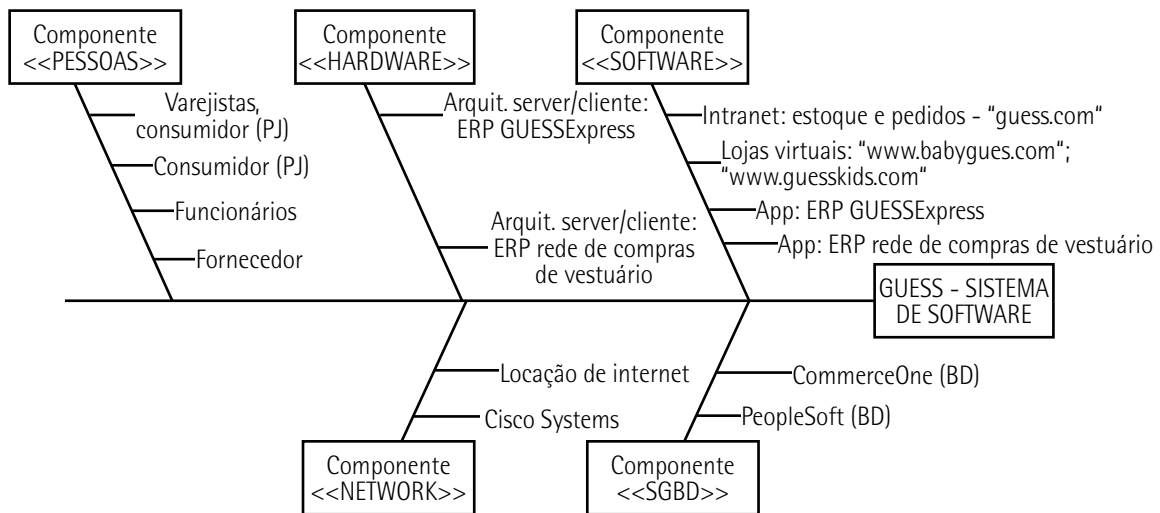


Figura 43 – Diagrama de causa e efeito aplicado na análise de sistema de software

Exemplo de aplicação

Estudo de caso: manutenção corretiva em módulo ERP de gerenciamento de armazém (WMS)

Em um módulo de gerenciamento de armazém (WMS) de um ERP logístico, utilizado para controlar o estoque e a separação de pedidos (picking), os operadores relataram que o sistema começou a alocar produtos para endereços de estoque incorretos. O algoritmo de alocação automática estava sugerindo posições que já estavam cheias ou que não eram adequadas para o tipo de produto (por exemplo: alocando produtos refrigerados em áreas de temperatura ambiente). Isso estava gerando atrasos nas ordens de separação e erros de inventário.

Ao identificar a falha, na manutenção corretiva inclui-se uma codificação de uma nova funcionalidade, que consiste na prática adicional de elaborar um relatório de exceção, de forma que o problema seja detectado inicialmente por alertas de exceção de inventário (uso, por exemplo, de uma KPI da logística), os quais fornecem uma notificação de alerta ou um sinal que dispara no painel de controle do WMS.

Por comparação, o software, ao tentar confirmar a nova alocação, fará a verificação de incompatibilidade entre pedido de alocação, alinhamento de tipo de produto com condições de armazenamento, locação ocupada e registro de status das condições para uso ou alerta de erro(s). Na ocorrência da incompatibilidade, um tíquete é liberado e categorizado como manutenção corretiva de prioridade alta (P2), pois impactava diretamente a eficiência operacional do armazém.

Manutenção preventiva

São ações proativas que visam evitar problemas futuros e aumentar a robustez do software. Inclui atividades como:

- análise de desempenho e melhorias;
- revisões de código (code review), especialmente orientadas por padrões de qualidade;
- atualizações regulares de segurança (patching);
- refatoração contínua para melhorar legibilidade, modularidade e manutenibilidade do código;
- verificação antecipada de riscos técnicos.

A prática de **refatoração**, recorrente em métodos ágeis como XP, reforça a sustentabilidade do software ao longo do tempo, mantendo o código limpo e resiliente a mudanças. Essa prática envolve a reestruturação e melhoria do código-fonte sem alterar seu comportamento externo. É o processo de fazer ajustes no código para torná-lo mais legível, eficiente, organizado e sustentável, sem afetar a funcionalidade do software.

Exemplo de aplicação

Estudo de caso 1: manutenção preventiva em redes social

O foco da manutenção preventiva em uma rede social complexa e distribuída está em garantir a disponibilidade, segurança e escalabilidade da plataforma antes que milhões de usuários sejam afetados.

1) Contexto: alto tráfego e dados distribuídos

Cenário: uma grande rede social lida com bilhões de interações (curtidas, postagens, mensagens) por dia, exigindo um sistema de computação em nuvem (cloud computing) altamente distribuído e complexo. O principal risco é a lentidão (lag) durante picos de tráfego (como após um grande evento global ou determinados períodos) ou ainda uma falha de segurança em massa (um cyber attack, por exemplo).

2) Práticas de manutenção preventiva aplicadas

Em vez de esperar por uma falha, as equipes de engenharia realizam ações contínuas como:

- **Inspeção contínua e refatoração de código (eficiência):** as equipes realizam auditorias periódicas no código que gerencia o feed de notícias, por exemplo, identificando consultas lentas ao banco de dados (inefficient database queries) ou loops de código complexos que, sob carga normal, funcionam bem, mas em picos de tráfego podem se tornar gargalos ou até mesmo gerar deadlocks. O deadlock (impasse, em português), chamado popularmente de "disputa de variáveis",

é onde dois ou mais processos ficam bloqueados indefinidamente, esperando uns pelos outros para liberar um recurso. Situação indesejável que ocorre em sistemas computacionais, especialmente em sistemas operacionais e bancos de dados.

- **Ação:** o código é refatorado para melhorar a eficiência algorítmica e melhorar a latência, antes que o aumento do tráfego cause um crash ou lentidão perceptível ao usuário.
- **Auditoria de segurança e caça a vulnerabilidades (security patching):** são inspeções realizadas de forma contínua e testes de penetração automáticos (pen-tests) nos serviços de autenticação e privacidade dos usuários.
 - **Ação:** ao identificar uma vulnerabilidade potencial, como uma falha de buffer overflow em um microserviço de mensagens, a equipe aplica o patch de segurança e atualiza as bibliotecas de terceiros (software subjacente) de forma coordenada e controlada, prevenindo, assim, um ataque de hackers que poderia comprometer milhões de contas no futuro.
- **Projeção de capacidade e otimização de infraestrutura (escalabilidade):** monitoramento constante pelas equipes de DevOps acompanhando o uso de recursos como CPU, memória, disco e rede, nos clusters de cache dos bancos de dados, usando essa análise de dados para projetar o crescimento futuro do tráfego.
 - **Ação:** antes que a utilização atinja um limite crítico –80%, por exemplo –, o time executa a expansão preventiva dos recursos de nuvem (scaling up ou scaling out), de forma a garantir que o sistema e software possam absorver o próximo pico de tráfego sem degradação do serviço, mantendo a disponibilidade e a agilidade.
- **Manutenção adaptativa:** a manutenção adaptativa envolve ajustes necessários para que o software continue operando adequadamente diante de mudanças no ambiente operacional, como atualizações de sistemas operacionais; mudanças em bibliotecas, frameworks ou drivers; novos requisitos legais ou regulatórios; integração com novos sistemas ou dispositivos; ou alterações arquiteturais em ambientes distribuídos ou em nuvem. A mudança que aciona a manutenção adaptativa não é um erro (corretiva) ou uma nova funcionalidade (evolutiva), mas sim uma adaptação regulatória e técnica ao ambiente externo.

Estudo de caso 2: manutenção adaptativa em tokens remotos e integração distribuída

Uma rede de varejo possui tokens remotos (pequenos quiosques ou leitores) espalhados em shopping centers e supermercados. Esses tokens, usados para consulta de preços e promoções pelos clientes, estão ligados a um sistema de gestão de preços centralizado (ERP) via rede. Contudo, devido a uma nova legislação de proteção de dados (LGPD ou GDPR equivalente), o sistema central precisou ser adaptado.

Ação: requisito de adaptação (regulamentação). A implementação da nova legislação exige que dados sensíveis de acesso e navegação (como o ID do token ou a localização exata da consulta) sejam

anonimizados ou criptografados antes de serem armazenados no banco de dados central, mesmo sendo dados não pessoais. O token não pode mais enviar dados de log em plain text (do português, texto puro, que é um conceito da segurança da informação que representa codificação de forma direta).

Em relação aos **tipos de manutenção** que ocorrem durante todo o ciclo de vida operacional do software, Pressman e Maxim (2021) e Sommerville (2016) estão em comum acordo quando afirmam que os tipos de manutenção do software são motivados por três tipos fundamentais, conforme o quadro a seguir.

Quadro 22 – Tipos de manutenção de software

Tipo de manutenção	Descrição
Manutenção corretiva	São as ações para identificar e reparar os defeitos no software. A correção de erros de codificação inclui falhas de lógica, erros de implementação ou comportamentos inesperados. Quanto mais cedo o erro é detectado no ciclo de vida, menor o custo de correção, sendo os erros de requisitos os mais dispendiosos por envolverem reanálise e reprojetos.
Manutenção preventiva	São melhorias estruturais realizadas antes que falhas ocorram. Envolve refatorações, otimizações, atualizações técnicas e ajustes que aumentam a confiabilidade e facilitam a evolução futura do software. É uma prática estratégica para reduzir riscos e custos ao longo do tempo.
Manutenção adaptativa	Refere-se às modificações necessárias para que o software se mantenha funcionando corretamente quando o ambiente externo muda. Abrange ajustes devido a novas versões de sistemas operacionais, bancos de dados, drivers, plataformas de nuvem ou integração com serviços externos.

Adaptado de: Pressman e Maxim (2021) e Sommerville (2016).

8.2 Evolução, integração e tendências do software

A evolução, integração e tendências do software refletem o movimento contínuo da indústria em direção a sistemas mais inteligentes, conectados e adaptáveis, prometendo moldar um futuro cada vez mais tecnológico e repleto de oportunidades para o desenvolvimento de soluções inovadoras.

A evolução do software ocorre por meio de constantes melhorias incrementais e inovações que ampliam desempenho, segurança e usabilidade, enquanto a integração garante a interoperabilidade entre plataformas, serviços e dispositivos distribuídos.

As tendências em engenharia de software ampliam o potencial de melhoria da experiência do usuário, eficiência operacional e segurança das aplicações. Tecnologias como inteligência artificial (IA), computação em nuvem, Internet das Coisas (IoT) e realidade virtual/aumentada estão transformando arquiteturas, modelos de interação e capacidades analíticas dos sistemas. A IA impulsiona automação e personalização, a nuvem possibilita escalabilidade e entrega contínua, a IoT expande ecossistemas digitais e as tecnologias imersivas redefinem interfaces. Em conjunto, esses avanços moldam o futuro do software, demandando práticas ágeis, seguras e orientadas a valor.

No desenvolvimento, tendências como arquiteturas baseadas em microsserviços, automação com IA generativa, observabilidade, desenvolvimento orientado a APIs e práticas DevSecOps transformam

profundamente o ciclo de vida das aplicações. Combinadas, essas forças formam o impulso para um ecossistema de software dinâmico, no qual a modernização constante e a capacidade de adaptação definem a competitividade e a sustentabilidade tecnológica das organizações.

8.2.1 Abordagens ágeis para manutenção e evolução

As abordagens ágeis aplicadas à manutenção e evolução de software surgem como resposta à necessidade de lidar com mudanças constantes, entregas contínuas e ciclos de adaptação rápidos. Em vez de tratar manutenção como uma fase isolada e reativa, a agilidade incorpora práticas que tornam o processo contínuo, incremental e colaborativo.

Frameworks como Scrum e Kanban promovem visibilidade do trabalho, priorização dinâmica de demandas e ciclos curtos de entrega, permitindo que a equipe responda rapidamente a defeitos, melhorias e novas necessidades de negócio. Além disso, métodos ágeis reforçam uma mentalidade de melhoria contínua, apoiada por feedback frequente e integração estreita entre desenvolvimento, operações e usuários. Outra característica fundamental dessas abordagens é o uso de práticas técnicas que reduzem riscos e aumentam a qualidade ao longo do tempo.

A CI/CD, o TDD e a automação de testes garantem que cada alteração seja validada rapidamente, minimizando retrabalho e encurtando o tempo entre o surgimento de uma demanda e sua implementação. A cultura DevOpsSec complementa esse cenário ao integrar desenvolvimento, operações e segurança, permitindo liberação de releases frequentes, sistemas mais confiáveis e ambientes padronizados, o que reduz significativamente o custo da manutenção corretiva, adaptativa e evolutiva.

Exemplo de aplicação

O uso do **Kanban** na manutenção de software pode ser observado em uma equipe responsável por garantir a estabilidade e a evolução contínua de um sistema corporativo. Nesse cenário, a equipe utiliza um quadro Kanban, como mostra a ilustração da figura 44, dividido em colunas como: Backlog, A fazer, Em progresso, Em testes, Aguardando homologação e Concluído.

As cores dos cartões, amplamente utilizadas, seguem níveis de criticidade: Vermelho – Crítico (P1 – Urgente); Laranja – Alto (P2 – Alto impacto); Amarelo – Médio (P3 – Impacto moderado); Verde – Baixo (P4 – Baixo impacto/melhoria); e Evolutivo/oportunidade – Azul (P5 – Melhoria futura).

Backlog	A fazer	Em progresso	Em testes	Aguardando homologação	Concluído
		 	 		 

Figura 44 – Modelo de quadro Kanban para acompanhamento de manutenção

Como mostra a figura a seguir, todas as demandas de manutenção corretiva, adaptativa ou evolutiva são registradas como cards, cada um representando uma atividade específica, como corrigir falha na API de autenticação, atualizar driver de banco de dados ou implementar novo campo no formulário de cadastro.

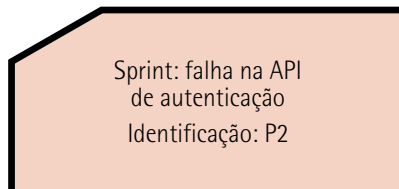


Figura 45 – Modelo cartão para quadro Kanban

8.2.2 Evolução do software

A evolução do software tem se voltado para práticas sistematizadas que integram desenvolvimento, manutenção e melhoria contínua em um fluxo único e incremental. A **gestão do backlog e os mecanismos de priorização**, juntamente com a definição clara dos papéis organizacionais, formam a base essencial para essa estrutura de trabalho.

O product owner, as equipes de desenvolvimento e os especialistas de domínio atuam de forma colaborativa para equilibrar RF, solicitações de evolução e necessidades operacionais. A priorização, fundamentada em critérios como valor de negócio, risco, impacto no usuário e custo da mudança orienta decisões estratégicas e garante que a evolução do software ocorra de maneira consistente com a dinâmica organizacional.



Saiba mais

Leia o artigo a seguir. Ele apresenta uma visão geral completa do contexto histórico da engenharia de software que vai até a época contemporânea dominada pela inteligência artificial (IA). Começando pelas linguagens básicas de programação de meados do século XX, o artigo destaca marcos importantes, incluindo o nascimento da programação orientada a objetos, a expansão das práticas de engenharia de software e o impacto transformador da programação estruturada.

JOSHUA, J. V.; ANDEME, D. A. N. A evolução da engenharia de software: do início pré-histórico à era da inteligência artificial. *IJCIT – International Journal of Computer and Information Technology*, v. 14, n. 1, mar. 2025. Disponível em: <https://tinyurl.com/4d57zxdk>. Acesso em: 30 dez. 2025.

Outro vetor relevante é a **integração contínua de defeitos e melhorias**, impulsionada tanto pelo DevOpsSec quanto por arquiteturas modulares e pipelines automatizados. A manutenção de um fluxo constante de correções, refatorações e incrementos reduz o acúmulo de defeitos latentes, favorece ciclos curtos de entrega e aumenta a confiabilidade do produto. O emprego de **métricas de fluxo**, como lead time e cycle time, constitui uma referência para quantificar a eficiência do processo, identificar gargalos e orientar decisões de capacidade e priorização. Essas métricas permitem que organizações operem com maior previsibilidade, ajustem rapidamente seus processos e sustentem um ritmo de entrega compatível com ambientes competitivos e de rápida mudança.

A **gestão da dívida técnica** e o **enquadramento ágil da manutenção contínua** despontam como elementos críticos para o futuro da evolução do software. A dívida técnica é entendida como o acúmulo de decisões de engenharia que prejudicam a manutenibilidade, escalabilidade ou qualidade, passando a ser tratada como um ativo monitorável, devendo ser visível no backlog e priorizada com base em seu impacto estrutural. A manutenção deixa de ser um conjunto de intervenções reativas e se consolida como uma prática integrada ao ciclo evolutivo do produto, sustentada por feedback contínuo, práticas de observabilidade, automação e melhoria incremental.

A evolução do software decorre, sobretudo, das constantes transformações nas necessidades de negócio e das falhas que emergem devido a mudanças no ambiente operacional. Esse processo deve ser analisado de forma integrada, considerando todos os componentes que compõem o sistema, os quais também evoluem em ritmos distintos.

Tais componentes incluem a lógica de processamento, a introdução de novas funcionalidades, as atualizações de hardware, a evolução do banco de dados e as alterações na infraestrutura de rede. Além disso, as mudanças solicitadas pelos usuários podem gerar impactos significativos, aumentando a complexidade operacional e, por consequência, o custo associado à evolução do software.

No ciclo de vida da evolução do software, os requisitos dos sistemas instalados mudam à medida que o negócio e o ambiente operacional evoluem, o que resulta na necessidade de novos releases que incorporam as modificações demandadas.

A evolução do software ocorre de maneira incremental e contínua, refletindo a natureza espiral da engenharia de software, na qual requisitos, design, implementação e testes se repetem ao longo de todo o ciclo de vida do sistema.

Em consonância com o **modelo espiral** (figura 46), o desenvolvimento avança por meio de versões sucessivas. Nas primeiras iterações, podem ser gerados modelos conceituais ou protótipos, enquanto nas iterações posteriores, são construídas versões cada vez mais completas e funcionais.

Dessa forma, as atualizações são produzidas em intervalos regulares, permitindo que o sistema incorpore gradualmente as mudanças necessárias conforme o negócio evolui, mantendo a qualidade e reduzindo riscos ao longo de cada ciclo.

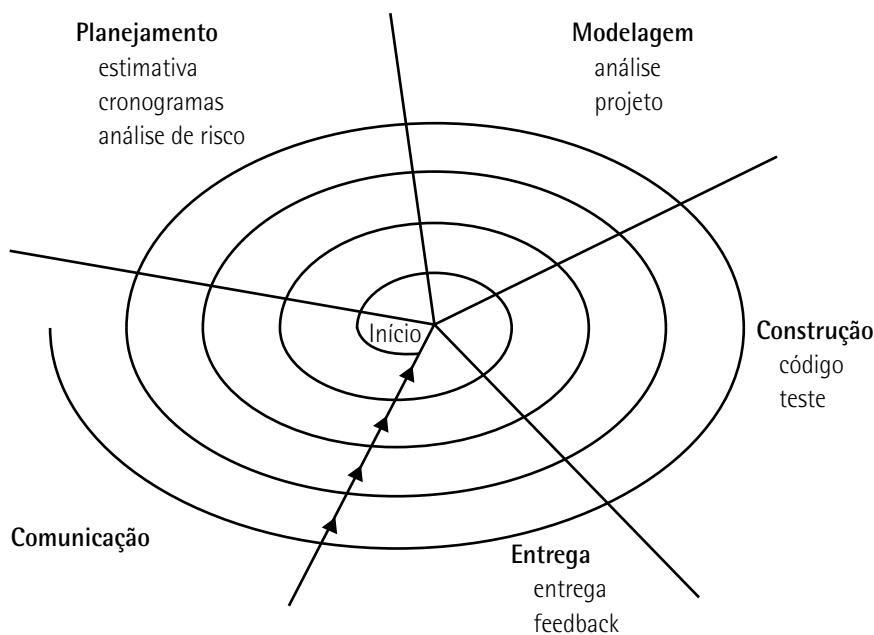


Figura 46 – Modelo espiral típico

Fonte: Pressman e Maxim (2021).

Na evolução do software, as propostas de mudança devem ser devidamente associadas aos componentes do sistema que serão afetados, permitindo avaliar com precisão o custo, o esforço e o impacto decorrentes da alteração. Essa atividade integra o processo de gerenciamento de mudanças, cuja responsabilidade inclui não apenas analisar e priorizar modificações, mas também garantir que as versões corretas dos artefatos de software sejam identificadas, controladas e incorporadas de forma consistente a cada release do sistema.

8.3 Estratégias estruturais: top-down e bottom-up

Para melhorar a compreensão do funcionamento do teste de software, especialmente no contexto de metodologias ágeis, é fundamental estabelecer duas perspectivas de interação distintas, que guiam a estratégia de testes:

- **Interface com o usuário (front-end/apresentação):** foco na experiência do usuário (UX), usabilidade, acessibilidade e validação de requisitos visuais e funcionais a partir do ponto de vista do cliente.
- **Interface com o ambiente operacional (backend/infraestrutura):** foco na interação do software com o sistema operacional, banco de dados, APIs de terceiros, desempenho, segurança e alocação de recursos.

Essas duas perspectivas informam o que deve ser testado, levando naturalmente às estratégias de integração **top-down** e **bottom-up**, relevantes mesmo em contextos ágeis, embora a sua aplicação seja adaptada.



Observação

Top-down significa de cima para baixo; bottom-up, de baixo para cima.

8.3.1 Top-down: integração orientada ao fluxo de valor

No desenvolvimento ágil, o objetivo principal é a entrega contínua de valor por meio de incrementos funcionais (sprints).

A **abordagem top-down** de testes de integração (como ilustrado na figura 47) verifica a integração dos componentes, começando pelos níveis superiores até os módulos subordinados de nível inferior. No desenvolvimento ágil, é uma estratégia usada para validar o fluxo de valor de ponta a ponta de forma rápida.

- **Ponto de partida:** inicia-se pelos requisitos dos módulos de alto nível, histórias de usuários e uso da interface, ou pelo módulo principal de controle, que lida com as regras de negócio de nível superior (por exemplo: característica de um checkout).
- **Stubs (simuladores):** são a validação da funcionalidade de ponta a ponta (end-to-end). Servem para que o módulo superior possa ser testado antes da conclusão dos módulos inferiores, ou seja, o foco é garantir que o fluxo de valor definido na user story (por exemplo: serviço de como o usuário pode efetuar um pagamento) funcione, mesmo que os componentes inferiores sejam inicialmente stubs.

Um stub é um substituto de um código que simula a saída de um módulo dependente. Na depuração do código, se consegue isso de forma fácil, alterando apenas as variáveis de entradas e saídas do trecho de código em teste.

- **Fluxo:** o foco é testar o fluxo de controle do sistema, caminhando da decisão (o que o sistema deve fazer) para o detalhe (como o código executa essa tarefa).

Embora o ágil priorize os testes unitários (bottom-up) para a qualidade do código, o top-down se manifesta nos testes de integração de serviços e testes de ponta a ponta (end-to-end), garantindo que as camadas superiores se comuniquem corretamente.

A técnica top-down verifica a integração dos componentes de um sistema, começando pelos níveis superiores (como requisitos, funções e interface do usuário) e seguindo, progressivamente, para os níveis inferiores (incluindo testes do código, respostas de interfaces e rede). O principal foco é que a informação (o resultado final) caminhe até os dados que a geraram no processamento, garantindo que o fluxo de controle seja validado desde o ponto de entrada.

Vejam, a seguir, uma representação da técnica top-down.

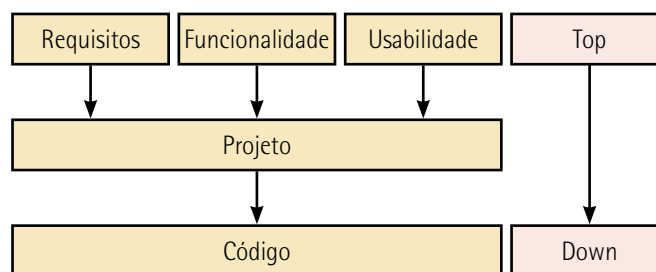


Figura 47 – A abordagem de teste top-down parte dos níveis superiores – no caso, em testes de requisitos, funcionalidade e usabilidade

8.3.2 Bottom-up: integração orientada ao componente

A **abordagem bottom-up** (ilustrada na figura 48) é mais comum no ágil moderno e verifica os componentes, começando pelos níveis mais baixos e granulares (código e módulos) antes de movê-los para cima. Ou seja, na estrutura bottom-up (o como tradicional) verifica-se a integração dos componentes da seguinte maneira:

- **Ponto de partida:** inicia-se pelos módulos atômicos ou de menor nível, que realizam funções específicas e detalhadas (exemplo: módulos de acesso a dados ou lógica de cálculo).
- **Fluxo de teste:** os módulos de nível inferior são combinados em clusters (conjuntos) e testados antes de serem integrados aos módulos de controle de nível superior.
- **Uso de drivers (controladores):** como os módulos de controle superiores ainda não foram integrados, o teste bottom-up utiliza drivers simuladores. Um driver simulador é um programa dummy que chama e controla o módulo que está sendo testado, fornecendo-lhe dados de teste. A função desse driver é, justamente, simular o ambiente de chamada do módulo superior para garantir que o módulo de baixo nível funcione corretamente sem interrupção.
- **Desdobramento:** os clusters de módulos testados (agora funcionando) substituem os drivers, e o processo continua subindo na hierarquia até que o módulo principal seja finalmente testado.

A abordagem bottom-up, ilustrada na figura a seguir (figura 48), é uma técnica que inicia os testes de integração pelos módulos de nível inferior, tipicamente em nível de código. Começando pela validação das funcionalidades mais granulares, como a interface com hardware, drivers e instalação, e seguindo, progressivamente, para os níveis superiores (níveis operacionais do sistema). O método bottom-up é, portanto, frequentemente aplicado nas fases iniciais do desenvolvimento de software para construir uma base de código robusta e verificada.

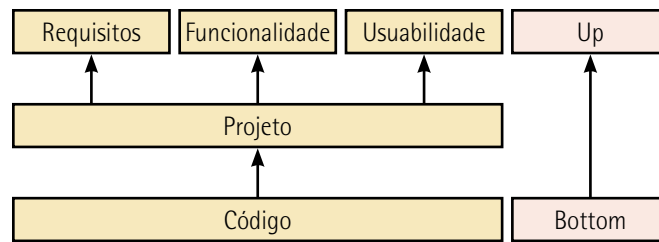


Figura 48 – A abordagem de teste bottom-up parte dos níveis inferiores a nível do código

8.4 Gestão de configuração do software

Os sistemas de software mudam continuamente durante o desenvolvimento e o uso. Bugs precisam ser corrigidos, requisitos evoluem e novas versões devem ser criadas. Além disso, atualizações de hardware e plataformas exigem adaptações, e funcionalidades introduzidas por concorrentes e usuários pressionam por melhorias. Assim, cada mudança gera uma nova versão, e a maioria dos sistemas passa a ser composta por múltiplas versões que precisam ser mantidas e gerenciadas.

A gestão de configuração (CM) trata das políticas, processos e ferramentas para gerenciar sistemas de software em constante mudança. É necessário gerenciar sistemas em evolução porque é fácil perder o controle sobre quais alterações e versões de componentes foram incorporadas em cada versão do sistema. As versões implementam propostas de mudança, correções de falhas e adaptações para diferentes hardwares e sistemas operacionais. (Sommerville, 2016, p. 731).

A **gestão de configuração do software (software configuration management – SCM)** controla diversas versões que podem estar em desenvolvimento e uso ao mesmo tempo. Se não possuir procedimentos eficazes de gestão de configuração, pode desperdiçar esforço modificando, por exemplo, a versão errada do sistema, entregar ao cliente o release incorreto ou até esquecer a localização do código-fonte de uma determinada versão do sistema ou do componente armazenado.

Sommerville (2016) sugere quatro principais atividades do gerenciamento de configuração, conforme o quadro a seguir:

Quadro 23 – Principais atividades do gerenciamento de configuração do software

Atividades	Descrição
Controle de versão	Envolve rastrear as múltiplas versões dos componentes do sistema e garantir que alterações feitas por diferentes desenvolvedores não interfiram umas nas outras
Gestão de mudanças	Consiste em registrar solicitações de mudança referentes ao software entregue por clientes e desenvolvedores, analisar custos e impactos dessas mudanças e decidir se e quando devem ser implementadas
Construção de sistemas	É o processo de reunir componentes de programa, dados e bibliotecas, compilando e vinculando esses elementos para criar um sistema executável
Gestão de releases	Envolve preparar o software para lançamento externo e manter o controle das versões do sistema que foram disponibilizadas para uso pelos clientes

Adaptado de: Sommerville (2016, p. 731).



Lembrete

A gestão de configuração de software (SCM) estabelece o controle sistemático dos artefatos do software ao longo de seu ciclo de vida, assegurando integridade, rastreabilidade e consistência entre versões, requisitos e entregas. Integrada às atividades de verificação e validação (V&V), a SCM permite confirmar que os itens de configuração foram corretamente implementados, testados e aprovados conforme os requisitos definidos, reduzindo riscos de inconsistências, regressões e falhas, além de fortalecer a qualidade e a confiabilidade do produto final.

8.4.1 Versionamento do software

As mudanças são inerentes a todo o ciclo de vida do software e tendem a se intensificar conforme o sistema cresce em complexidade. Sem um controle adequado, essas alterações podem gerar inconsistências, retrabalho e falhas na entrega do produto ou de seus componentes. O **controle de versões** alinhado ao **controle de releases** formam os processos responsáveis por registrar, organizar e acompanhar a evolução dos artefatos do software ao longo do tempo, possibilitando a rastreabilidade das modificações realizadas. Esse processo, conhecido como **versionamento**, permite identificar claramente cada estado evolutivo do sistema.

No contexto da engenharia de software, **versão** refere-se a uma configuração específica do software que incorpora novas funcionalidades, correções ou adaptações, e que pode estar destinada ao uso interno para desenvolvimento, testes ou integração. Já o **release** é uma versão formalmente preparada e estabilizada para distribuição ao cliente ou implantação em ambiente de produção, seguindo critérios de qualidade e aprovação definidos pela equipe.



Observação

O versionamento de software desempenha papel essencial na coordenação e no controle das mudanças ao longo do ciclo de desenvolvimento, estando diretamente alinhado às atividades de testes unitários, de integração, de aceitação e de regressão.

Cada nova versão ou incremento do sistema deve estar associada a evidências de testes executados, garantindo que alterações isoladas sejam validadas individualmente, que as integrações entre componentes funcionem corretamente, que os requisitos do usuário sejam atendidos e que funcionalidades previamente implementadas não sejam impactadas negativamente.

Dessa forma, o versionamento aliado aos diferentes níveis de teste assegura rastreabilidade, estabilidade evolutiva e maior confiabilidade das entregas de software.

O quadro a seguir ilustra que o controle de versões utiliza técnicas e convenções de identificação de componentes que suportam a integração segura de mudanças, permitindo que múltiplos desenvolvedores trabalhem paralelamente sem comprometer a integridade do sistema.

Quadro 24 – Técnicas básicas de numeração e identificação da versão aplicadas no gerenciamento de versões

Técnicas básicas	Descrição
Numeração de versões (version numbering/semantic versioning)	É o esquema de identificação mais comum. Atribui-se um número, explícito e único, de versão ao componente. Em práticas ágeis, costuma seguir o modelo major.minor.patch, facilitando integrações frequentes e rápida compreensão do impacto da alteração Exemplo: Versão 3.21 – 3 representa mudança estrutural significativa (major), 2 indica incremento funcional (minor), e 1 refere-se à correção de defeitos ou pequenos ajustes (patch)
Identificação baseada em atributos (attribute-based identification)	Cada componente recebe um identificador associado a atributos específicos (tipo, funcionalidade, módulo, variante, sprint ou branch). Essa estratégia é útil em ambientes ágeis com múltiplos times ou domínios, permitindo organização modular e rastreabilidade refinada Exemplo: Versão TEC01_1 – TEC – representa o módulo teclado, 01 inserção de um mapa de caracteres (incremento funcional) e _1 indica ajuste ou correção
Identificação orientada a mudanças (change-oriented identification)	Relaciona explicitamente a versão a uma ou mais solicitações de mudança, histórias de usuário ou itens do backlog. Em métodos ágeis, facilita o rastreamento direto entre commits, tarefas e entregáveis Exemplo: CAR_04 – mudança associada à quarta solicitação registrada por "Carlos" ou ao item 04 do backlog do módulo correspondente
Identificação vinculada ao pipeline ágil (commit/build tagging)	Utilizada em DevOps e integração contínua. Cada alteração é identificada por tags, hashes de commit (exemplo: Git) ou identificadores automáticos do pipeline. Garante rastreabilidade completa da origem da mudança e acelera correções Exemplo: build-2025.04.15-#842 ou commit a7f9c2b
Versões orientadas à entrega (release-based identification)	Identificadores próprios para releases estáveis, úteis em modelos ágeis com entregas frequentes. Podem ser alinhados ao planejamento de sprints, releases trimestrais ou marcos do produto Exemplo: Release 2025.Q1, Sprint-12-Release

Exemplo de aplicação

Estudo de caso: controle de versão baseada em numeração de versões – sistema Android

Padrão de numeração de versão: X.Y.Zzzz, onde:

- **X:** Corresponde a uma mudança de estrutura do software.
- **Y:** Inserção ou adaptação de novas funções ou funcionalidade.
- **Zzzz:** Adaptações, correções ou implementações no código fonte.

"Estou agora verificando meu smartphone com Android. Hoje, dia 20/11/2025, observo em "Configurações / Sobre o telefone" em versão do kernel 5.4.242". Aplicando o conceito de numeração de versões, chegamos à seguinte análise:

- **X = 5:** indica a 5ª geração da arquitetura do Android, com mudanças estruturais no sistema, no modelo de permissões e nas APIs principais.

- **Y = 4:** indica a inserção ou adaptação de novas funcionalidades incrementais incorporadas à versão principal do kernel do Android 5, como melhorias no desempenho gráfico e ajustes no gerenciamento de energia.
- **Zzz = 242:** representa correções no código-fonte, incluindo ajustes de estabilidade, atualizações de segurança e reparos de vulnerabilidades reportadas.

Sempre existem mais versões de um sistema do que releases, pois nem toda versão é destinada ao uso externo. Um **release** é uma versão formal do software autorizada para distribuição ao cliente, resultado de um processo controlado que valida estabilidade, segurança e completude funcional. A preparação de um release envolve uma série de elementos técnicos e organizacionais, tais como: instaladores devidamente empacotados, manuais técnicos e de usuário revisados, arquivos de configuração padronizados, além de bibliotecas, conjuntos de dados e scripts de registro necessários para garantir sua correta instalação e operação no ambiente do cliente.

Exemplo de aplicação

Estudo de caso: controle de versão baseada em grafo de controle

Considere o grafo de fluxo de versões ilustrado na figura a seguir, no qual as versões internas do sistema (V10.0, V1.1, V1.31, etc.) representam ciclos incrementais de desenvolvimento realizados por diferentes equipes. Cada nó do grafo indica um estado evolutivo do código-fonte, enquanto os ramos mostram bifurcações criadas para correção emergencial de defeitos ou desenvolvimento paralelo de funcionalidades. Os releases (Rel 1.0 e Rel 1.1), por sua vez, representam versões estabilizadas e aprovadas para distribuição ao cliente.

A numeração do release não depende diretamente da sequência de versões internas; entretanto, neste exemplo, o primeiro dígito do release reflete uma mudança majoritária na arquitetura do produto, enquanto o dígito subsequente representa a consolidação das funcionalidades agregadas no ciclo de desenvolvimento anterior.

Dessa forma, o grafo permite visualizar, de forma clara, o fluxo evolutivo do sistema, evitando conflitos de integração e assegurando rastreabilidade entre versões, correções e funcionalidades concluídas.

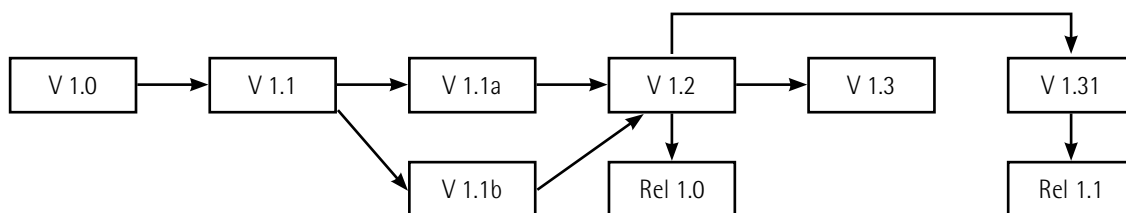


Figura 49 – Modelo de grafo de controle aplicado na numeração de versões e releases

8.5 Métodos, ferramentas e técnicas (MF&T): controle do código-fonte com GitHub

O **GitHub** é a maior plataforma de hospedagem de código-fonte e gerenciamento de projetos de software do mundo. Embora seja construído sobre o sistema de controle de versão Git, ele vai além de apenas armazenar código, atuando como uma rede social e uma ferramenta de colaboração completa para desenvolvedores.

O principal propósito do GitHub é facilitar a colaboração e o controle de versão de projetos de software. Basicamente ele serve para:

- **Hospedagem de repositórios:** armazenar o código-fonte de projetos em um local central e seguro na nuvem, acessível por qualquer pessoa autorizada.
- **Colaboração distribuída:** possibilitar que várias pessoas (equipes ou comunidades open source) trabalhem no mesmo código simultaneamente, sem interferir umas nas outras.
- **Controle de versão:** usar o Git para rastrear todas as alterações feitas no código, permitindo que a equipe volte a qualquer versão anterior, se necessário.
- **Gerenciamento de projetos:** oferecer ferramentas para planejar, rastrear e documentar o desenvolvimento do software.

O GitHub oferece um conjunto robusto de ferramentas que dão suporte a todo o ciclo de vida do desenvolvimento de software, desde a hospedagem até o controle de versão. O quadro a seguir apresenta os principais recursos do GitHub.

Quadro 25 – Funcionalidades do GitHub

Funcionalidade	Descrição
Repositórios (repos)	Onde todo o código, histórico (commits) e arquivos do projeto são armazenados. Podem ser públicos (visíveis para todos) ou privados. Cada commit representa uma mudança no código
Branches (ramificações)	Permite que desenvolvedores criem cópias isoladas do código principal (main) para trabalhar em novas funcionalidades ou correções sem afetar a versão estável. O trabalho com branches (ramificações) permite manter múltiplas linhas de evolução do software, exatamente como em um grafo de versões. Os exemplos mais comuns são: main (versão estável do sistema), develop (linha de integração, modelo GitFlow; feature/ (novas funcionalidades), fix/ (correção de erros) e release/ (preparação de releases)
Pull requests (PRs)	É um mecanismo central de colaboração para integrar código entre branches. É o pedido formal para integrar as mudanças de um branch de volta ao branch principal. Permite revisão de código antes da fusão
Revisão de código	Permite que outros membros da equipe revisem, comentem e sugiram alterações no código dentro de um pull request antes que ele seja aceito

Prática de inicialização do GitHub:

Etapa 1 – Criando o repositório

- Acesse <https://github.com>.
- Cadastre-se.

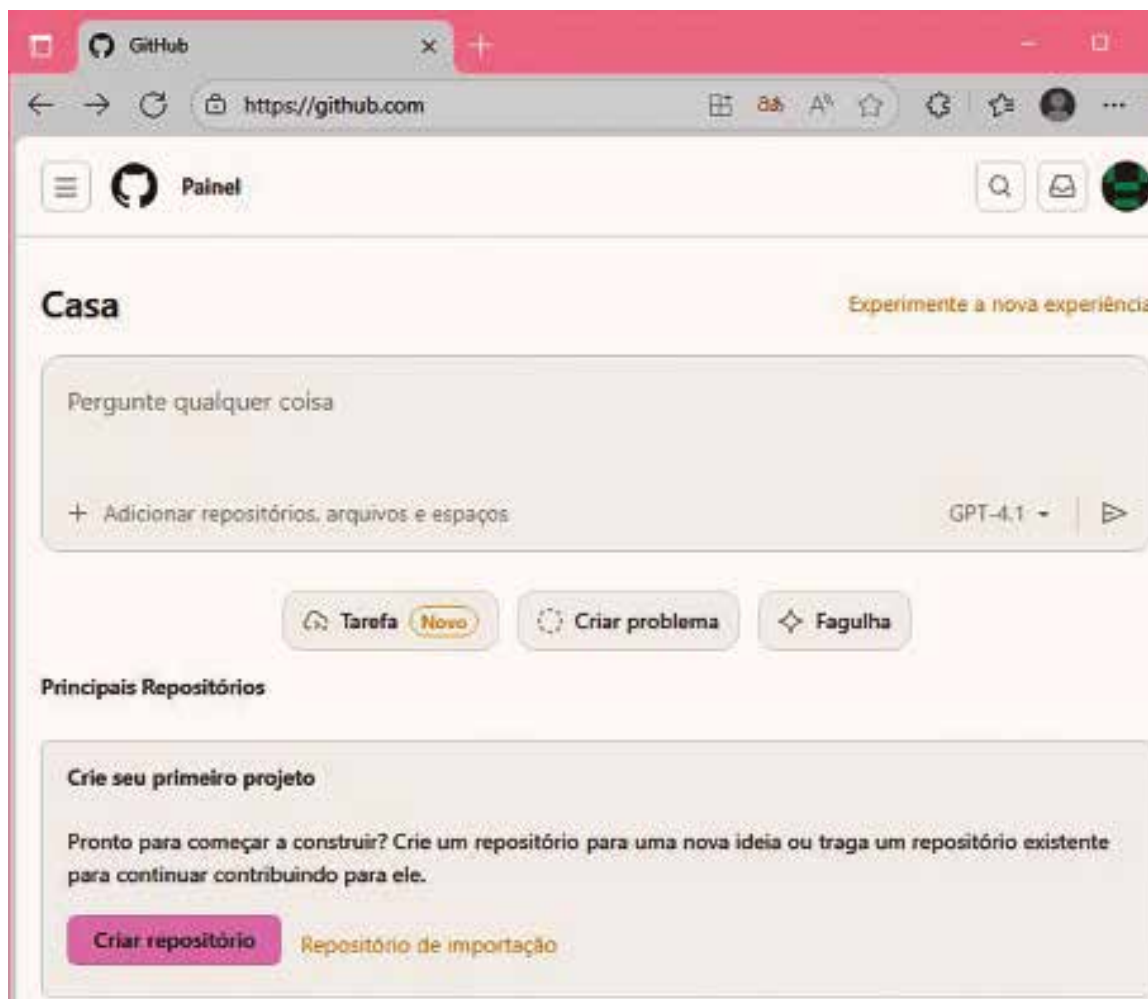
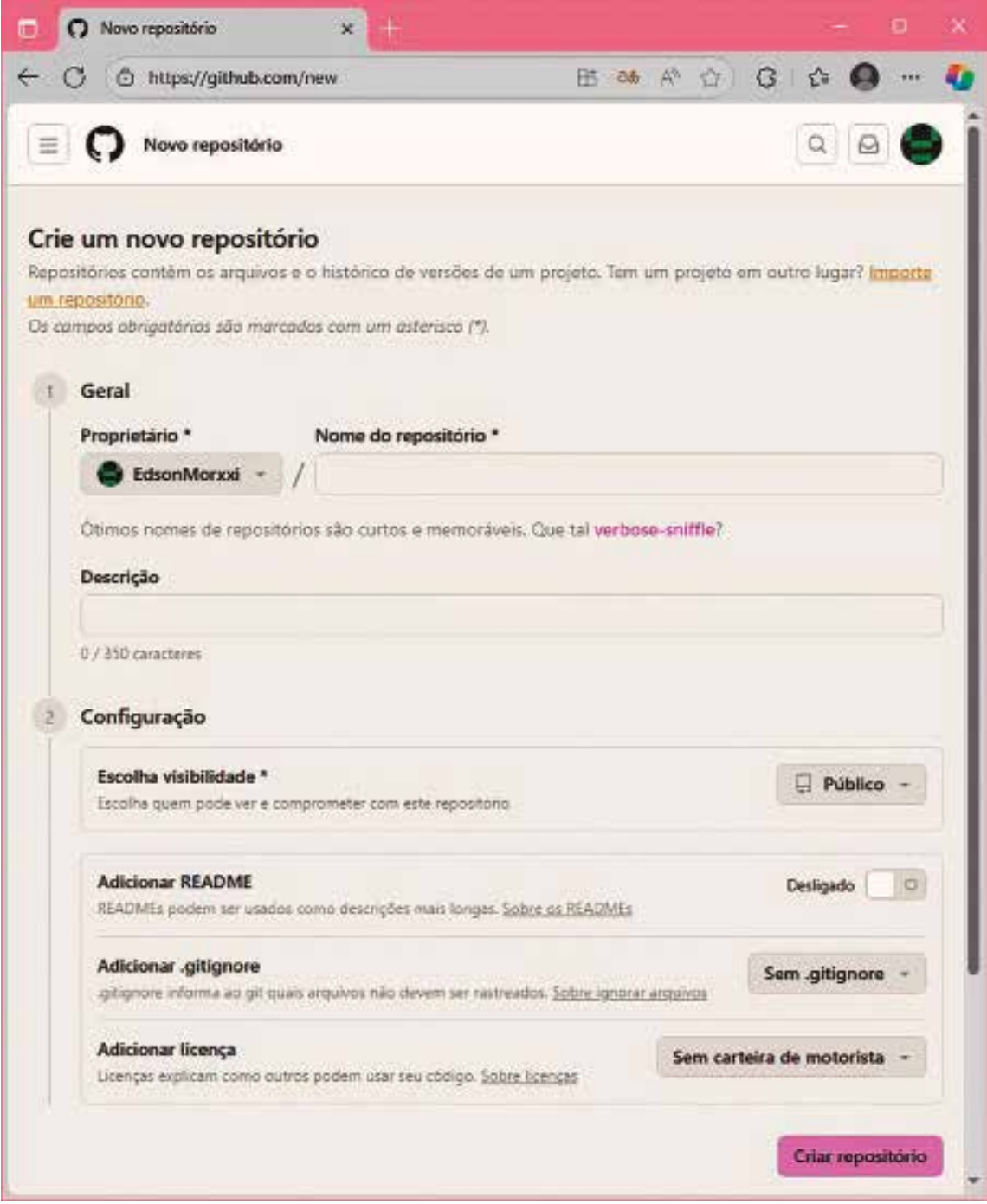


Figura 50 – GitHub: tela inicial

- Clique em criar repositório.



The screenshot shows the GitHub 'Novo repositório' (New repository) page. The browser address bar shows 'https://github.com/new'. The page title is 'Crie um novo repositório'. Below the title, there is a brief explanation of repositories and a link to 'Import a repository'. The page is divided into two sections: '1 Geral' and '2 Configuração'. In the 'Geral' section, there are fields for 'Proprietário' (selected as 'EdsonMorxxi') and 'Nome do repositório'. Below these is a 'Descrição' field with a character count '0 / 350 caracteres'. The 'Configuração' section includes options for 'Escolha visibilidade' (set to 'Público'), 'Adicionar README' (set to 'Desligado'), 'Adicionar .gitignore' (set to 'Sem .gitignore'), and 'Adicionar licença' (set to 'Sem carteira de motorista'). A 'Criar repositório' button is at the bottom right.

Novo repositório

https://github.com/new

Crie um novo repositório

Repositórios contêm os arquivos e o histórico de versões de um projeto. Tem um projeto em outro lugar? [Importe um repositório](#).

Os campos obrigatórios são marcados com um asterisco (*).

1 Geral

Proprietário * **Nome do repositório ***

EdsonMorxxi /

Ótimos nomes de repositórios são curtos e memoráveis. Que tal `verbose-sniffle`?

Descrição

0 / 350 caracteres

2 Configuração

Escolha visibilidade * **Público**

Escolha quem pode ver e comprometer com este repositório.

Adicionar README **Desligado**

READMEs podem ser usados como descrições mais longas. [Sobre os READMEs](#)

Adicionar .gitignore **Sem .gitignore**

.gitignore informa ao git quais arquivos não devem ser rastreados. [Sobre ignorar arquivos](#)

Adicionar licença **Sem carteira de motorista**

Licenças explicam como outros podem usar seu código. [Sobre licenças](#)

Criar repositório

Figura 51 – GitHub: crie um novo diretório

- Dê o nome do repositório e faça uma breve descrição (fluxo de versões).



Figura 52 – GitHub: criar espaço de código

Etapas 2 – Uso do terminal:

- Na área do terminal, crie em um novo espaço de código a seguinte sequência de códigos:

Quadro 26

<code>git init</code>	; Inicia um novo repositório Git com local no diretório
<code>git add .</code>	; Adiciona todos os arquivos novos, modificados e excluídos do diretório de trabalho para a área de staging
<code>git commit -m "Versões e Releases"</code>	; Grava as alterações preparadas na área de staging no histórico do repositório local
<code>git push -u origin main</code>	; Envia os commits locais para o repositório remoto

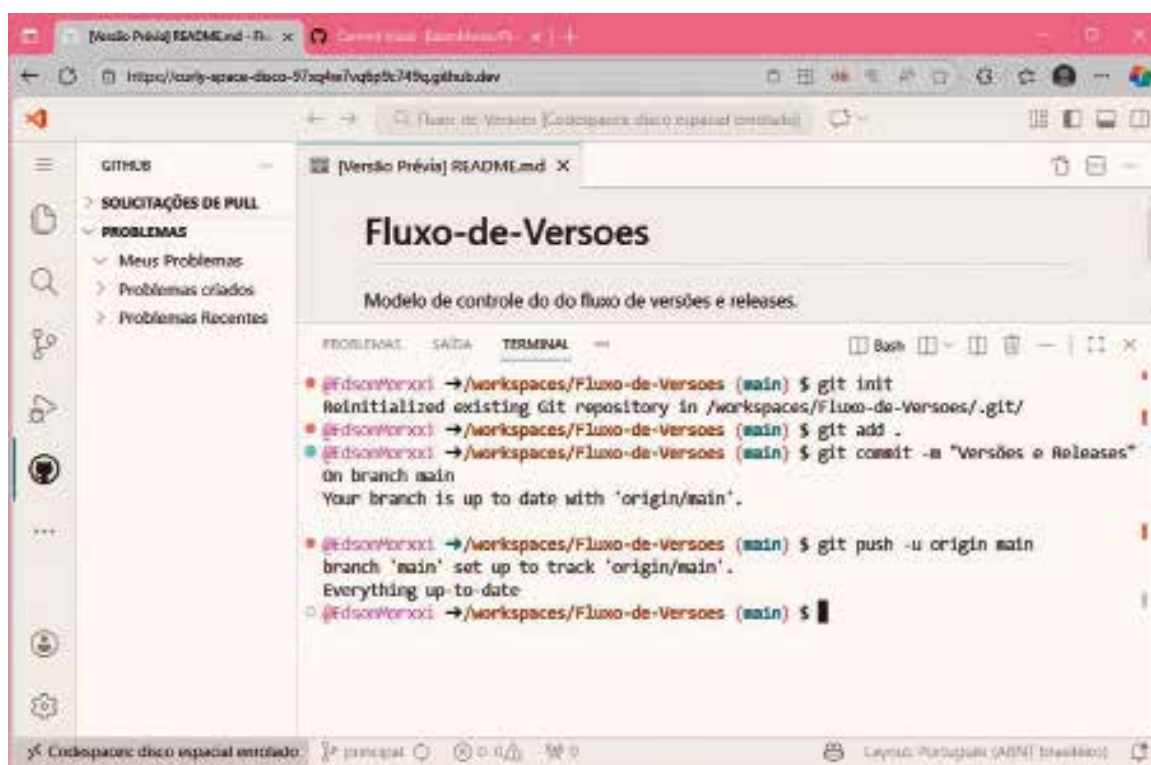


Figura 53 – GitHub: terminal

Etapas 3 – Commit inicial

Finalmente já se pode começar o projeto. A tela a seguir (figura 54) trata da interface do GitHub Codespaces + VS Code Web, mostrando o resultado de um commit inicial e o envio (push), justamente, para o GitHub.

Neste momento, são observados as seguintes áreas de estados:

- Área de ALTERAÇÕES (lado esquerdo):
 - Aqui o VS Code lista todos os arquivos modificados no repositório.

- Aparece um arquivo chamado README.md. Ele está na seção "Alterações". Isso indica que esse arquivo foi alterado, está modificado e ainda não havia sido commitado, e logo acima há um campo Mensagem do commit, onde você digita algo como "Versões e Releases".
- O painel central mostra o arquivo README.md, que indica duas linhas que foram adicionadas ao arquivo.
- O histórico de commits é apresentado em um pop-up (janela flutuante) sobreposto à área do terminal.
 - **Autor:** EdsonMoroxxi; horário do commit (38 minutos atrás); mensagem do commit: "Commit inicial".
 - **Informação do que foi modificado:** 1 arquivo alterado e 2 inserções. No caso, foram adicionadas duas linhas no README.md.

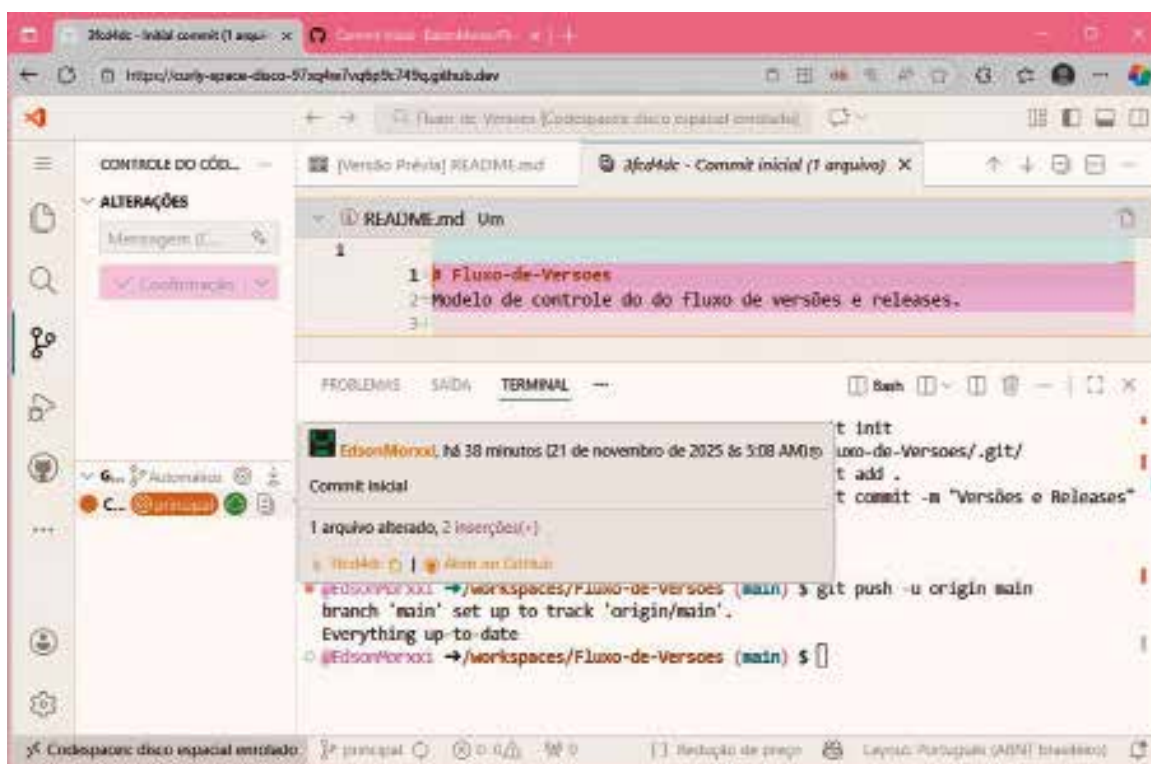


Figura 54 – GitHub: commit inicial



Resumo

Esta unidade abordou, de maneira integrada, os fundamentos que sustentam a qualidade do software ao longo de todo o seu ciclo de vida, unindo práticas de verificação & validação (V&V) e processos de gestão de mudanças e evolução. A primeira parte da unidade se concentrou nos princípios de V&V, cujo objetivo é garantir tanto a correção interna do software quanto sua aderência às necessidades do usuário. Foram discutidas as técnicas de verificação do código, com foco em depuração e diagnóstico de falhas, que permitem identificar e corrigir defeitos ainda nas fases iniciais do desenvolvimento. Em seguida, foram apresentados os fundamentos dos testes de software, destacando seus objetivos, abordagens e boas práticas que orientam a elaboração de cenários de teste e critérios de aceitação.

A unidade também explorou as abordagens caixa-preta e caixa-branca, enfatizando como cada uma contribui para uma compreensão complementar da qualidade: enquanto a caixa-preta analisa o comportamento externo do sistema, a caixa-branca examina sua lógica interna e estrutura. Além disso, foram detalhados os principais níveis de teste: unitário, de integração, de aceitação e de regressão, que estruturam o processo de avaliação contínua do software, desde elementos isolados até o comportamento global do sistema. Nesse contexto, o método test-driven development (TDD) foi apresentado como uma estratégia orientada à qualidade que antecipa os critérios de teste antes mesmo da implementação, fortalecendo o design do código. A etapa final dessa primeira parte abordou os testes alfa e beta, que envolvem validações internas e externas realizadas antes da liberação oficial do software.

A segunda parte da unidade tratou da gestão de mudanças e da evolução do software, reconhecendo que sistemas estão em constante transformação e que essa dinâmica exige processos de controle, análise e planejamento. Foram discutidos os fundamentos da gestão de mudanças, com ênfase em práticas que permitem avaliar impactos, priorizar solicitações e manter a rastreabilidade das alterações ao longo do tempo. A unidade também explorou a evolução do software sob diferentes perspectivas, destacando abordagens ágeis que favorecem ciclos curtos, entregas incrementais, integração contínua e orientação ao valor de negócio.

As estratégias estruturais de desenvolvimento, como top-down e bottom-up, foram apresentadas como alternativas complementares para lidar com a integração de componentes e fluxos de valor em arquiteturas cada vez mais complexas. A gestão de configuração do software ocupa

posição central nesse processo, uma vez que garante o controle de artefatos, a organização de versões e a consistência entre os elementos que compõem um sistema. Nesse contexto, o versionamento é discutido como prática fundamental para manter a integridade e a rastreabilidade das mudanças.

A unidade conclui com a aplicação prática de métodos, ferramentas e técnicas (MF&T), apresentando o uso do GitHub como ferramenta moderna para controle distribuído do código-fonte, colaboração entre equipes e automação de processos de integração e liberação.

Dessa forma, a unidade ofereceu uma visão completa do ciclo de qualidade no desenvolvimento de software, combinando técnicas de verificação, práticas de teste, mecanismos de controle de mudanças e estratégias de evolução contínua, garantindo que o produto final seja confiável, sustentável e alinhado às necessidades do usuário e do mercado.



Exercícios

Questão 1. Uma equipe está desenvolvendo, em sprints, um módulo de cadastro de pacientes. Ela quer:

- 1) confirmar, continuamente, que o módulo está sendo construído conforme os requisitos, as especificações e os padrões técnicos;
- 2) demonstrar, a cada entrega, que o que foi entregue atende ao propósito real de uso e às necessidades do usuário.

Em um contexto ágil, qual alternativa a seguir descreve corretamente esses dois focos (1 e 2) e exemplifica práticas associadas ao segundo foco (2)?

- A) O foco 1 é a verificação e o foco 2 é a validação. No segundo foco, a equipe pode usar as sprint reviews com stakeholders, a prototipação rápida, os testes de aceitação orientados pelo cliente (ATDD) e a definição de pronto (DoD) como critério de aceite e de demonstração.
- B) O foco 1 é a validação e o foco 2 é a verificação. No segundo foco, a equipe pode usar as sprint reviews com o cliente, os testes de aceitação (ATDD), os testes de usabilidade com usuários e um piloto controlado em ambiente representativo para demonstrar a aderência ao uso real.
- C) O foco 1 e o foco 2 são sinônimos. Assim, a equipe pode tratar ambos com as sprint reviews, os testes de aceitação (ATDD), os testes com usuários (por tarefas) e as simulações de uso em cenários reais, pois isso já cobre toda a avaliação necessária.
- D) O foco 1 é resolvido exclusivamente por depuração (debug) e o foco 2 é resolvido exclusivamente por testes automatizados de aceitação (ATDD). Para demonstrar o segundo foco, bastam as Sprint Reviews e a prototipação, dispensando-se a participação de usuários e de stakeholders.
- E) O foco 1 deve ocorrer apenas no fim do projeto e o foco 2 pode ser tratado apenas ao final, por meio de uma Sprint Review final, de testes de aceitação (ATDD), de uma sessão de testes de usabilidade e de um piloto curto, porque a conformidade com o requisito já é suficiente para afirmar a adequação ao usuário.

Resposta correta: alternativa A.

Análise das alternativas

A) Alternativa correta.

Justificativa: o foco 1 corresponde à verificação (conferir conformidade com os requisitos, com as especificações e com os padrões técnicos). O foco 2 corresponde à validação (demonstrar que o incremento atende ao propósito de uso e às necessidades do usuário). As práticas citadas (por exemplo, as Sprint Reviews com stakeholders, os testes de aceitação orientados pelo cliente e as demonstrações baseadas em critérios de aceite) são compatíveis com o segundo foco em um fluxo incremental de entregas.

B) Alternativa incorreta.

Justificativa: embora os exemplos apresentados (as sprint reviews com o cliente, os testes de aceitação e os testes de usabilidade) sejam coerentes com o segundo foco, a alternativa inverte os conceitos ao afirmar que o foco 1 é a validação e o foco 2 é a verificação.

C) Alternativa incorreta.

Justificativa: apesar de listar práticas ligadas ao segundo foco, a alternativa é incorreta ao declarar que a verificação e a validação são sinônimas. Os focos são distintos: um trata da conformidade com requisitos e padrões e o outro trata da adequação ao uso e às necessidades do usuário.

D) Alternativa incorreta.

Justificativa: a alternativa erra ao reduzir o foco 1 à depuração "exclusiva" e ao reduzir o foco 2 à automação "exclusiva". Além disso, ela entra em contradição ao afirmar que a validação pode ser demonstrada por Sprint Reviews e pelos protótipos, mas ao mesmo tempo "dispensa" a participação de stakeholders e de usuários, que é justamente o eixo de demonstração do segundo foco.

E) Alternativa incorreta.

Justificativa: embora cite práticas ligadas ao segundo foco, a alternativa falha ao deslocar a verificação e a validação para "apenas o fim do projeto" e ao sugerir que a conformidade com requisitos é suficiente para concluir adequação ao usuário. A validação depende de evidências de uso e de aceitação e tende a ser mais efetiva quando ocorre a cada entrega.

Questão 2. Durante uma sprint, um time detectou um defeito intermitente: o sistema congela quando duas ações ocorrem quase ao mesmo tempo. O time decide executar um processo de depuração (debugging) em quatro tarefas principais, incluindo o uso de breakpoints para observar o comportamento do código e, ao final, ações para evitar recorrência do problema (por exemplo, checar os efeitos colaterais em outras partes do sistema e rever as entradas e as saídas). Qual alternativa apresenta a sequência correta dessas tarefas com as descrições compatíveis?

A) Diagnóstico (Diagnose) → Reprodução (Reproduce) → Reflexão (Reflect) → Correção (Fix).

B) Reprodução (Reproduce) → Diagnóstico (Diagnose) → Correção (Fix) → Reflexão (Reflect).

C) Correção (Fix) → Reprodução (Reproduce) → Diagnóstico (Diagnose) → Reflexão (Reflect).

D) Reflexão (Reflect) → Correção (Fix) → Diagnóstico (Diagnose) → Reprodução (Reproduce).

E) Reprodução (Reproduce) → Correção (Fix) → Diagnóstico (Diagnose) → Reflexão (Reflect).

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: a alternativa coloca o diagnóstico antes da reprodução. A reprodução é o passo que viabiliza localizar o ponto do defeito e sustenta o diagnóstico.

B) Alternativa correta.

Justificativa: As quatro tarefas são, em ordem: (1) reproduzir o erro em condições controladas (incluindo breakpoints), (2) diagnosticar a causa, a gravidade, a prioridade e os riscos, (3) corrigir e testar (com apoio de práticas como os testes automatizados, a integração contínua e o TDD) e (4) refletir e aplicar medidas para evitar a recorrência, verificando os impactos em outras partes do sistema e revisando as entradas e as saídas.

C) Alternativa incorreta.

Justificativa: corrigir antes de reproduzir e diagnosticar contraria a sequência: a correção vem após localizar e compreender o defeito.

D) Alternativa incorreta.

Justificativa: a reflexão é descrita como etapa posterior à correção, com foco em prevenir recorrência e checar efeitos colaterais; não é etapa inicial.

E) Alternativa incorreta.

Justificativa: a alternativa antecipa a correção para antes do diagnóstico; ele é o passo em que se avaliam a causa, a gravidade, a prioridade e os riscos, antes de implementar a solução.

REFERÊNCIAS

ARQUITETURA do cluster: os conceitos arquiteturais por trás do Kubernetes. *Kubernetes*, 2025. Disponível em: <https://tinyurl.com/2edbr9c9>. Acesso em 23 dez. 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). *NBR ISO/IEC 9126-1 – Engenharia de software – Qualidade de produto*. Rio de Janeiro, 2003.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). *NBR ISO/IEC 9241-11 – Requisitos ergonômicos para trabalho de escritórios com computadores*. Rio de Janeiro, 2002.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). *NBR ISO/IEC 12207 – Tecnologia de informação – Processos de ciclo de vida de software*. Rio de Janeiro, 1998.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). *NBR ISO/IEC 14598-1 – Tecnologia de informação – Avaliação de produto de software*. Rio de Janeiro, 2001.

ASANOME, C. et al. *Guia para utilização das normas sobre avaliação de qualidade de produto de software: NBR ISO/IEC 9126 e NBR ISO/IEC 14598*. Curitiba: ABNT, 1999.

BAHN, C. Prática recomendada do IEEE para especificações de requisitos de software. *IEEE*, 1998. Disponível em: <https://tinyurl.com/2rf6uv8j>. Acesso em: 30 dez. 2025.

BEZERRA, E. *Princípios de análise e projetos de sistemas com UML*. Rio de Janeiro: Campus-Elsevier, 2003.

CAMARGO, R. Extreme Programming: quais principais regras e valores? *Robson Camargo Projetos e Negócios*, [s.d.]. Disponível em: <https://tinyurl.com/mr2d9xjh>. Acesso em: 30 dez. 2025.

CATEGORIA de modelagem de negócios: especificações associadas. *OMG – Object Management Group®*, [s.d.]. Disponível em: <https://tinyurl.com/4a6455me>. Acesso em: 30 dez. 2025.

CICLO PDCA e ISO 9001: qual a relação? Frons, 14 jan. 2020. Disponível em: <https://tinyurl.com/2mxknz6b>. Acesso em: 18 dez. 2025.

CMMI®. *CMMI® for systems engineering/software engineering, version 1.02: staged representation*. Pittsburgh: CMMI® Carnegie Mellon – Software Engineering Institute (SEI), 2000.

CMMI®. *CMMI® para desenvolvimento – versão 1.2: melhoria de processos visando melhores produtos*. Pittsburgh: CMMI® Carnegie Mellon – Software Engineering Institute (SEI), 2006.

COMEÇANDO – uma breve história do Git. *GIT-SCM*, [s.d.]. Disponível em: <https://tinyurl.com/thbnc4h9>. Acesso em: 16 dez. 2025.

CÔRTEZ, M. L. *Modelos de qualidade de software: ISO 15504*. São Paulo: IC-Unicamp, 2017.

FIORINI, S.; STAA, A.; BAPTISTA, R. M. *Engenharia de software com CMM*. Rio de Janeiro: Brasport, 1998.

FOURNIER, R. *Desenvolvimento e manutenção de sistemas estruturados*. São Paulo: Makron Books, 1994.

FUGITA, H. S.; CARVALHO JÚNIOR, D. S. Método de análise e projetos em SOA. *DevMedia*, 2010. Disponível em: <https://tinyurl.com/4y8ftzdf>. Acesso em: 22 dez. 2025.

HUMPHREY, W. S. *A discipline for software engineering*. Reading: Addison-Wesley, 1995.

JOSHUA, J. V.; ANDEME, D. A. N. A evolução da engenharia de software: do início pré-histórico à era da inteligência artificial. *IJCIT – International Journal of Computer and Information Technology*, v. 14, n. 1, mar. 2025. Disponível em: <https://tinyurl.com/4d57zxdk>. Acesso em: 30 dez. 2025.

KOSCIANSKI, A. *Qualidade de software: aprenda as metodologias e técnicas mais modernas para o desenvolvimento de software*. 2. ed. São Paulo: Novatec, 2007.

MANN, S. All about COBIT. *ManageEngine*, 2019. Disponível em: <https://tinyurl.com/288hw9uj>. Acesso em: 30 dez. 2025.

MELHORIA do Processo de Software Brasileiro. *Softex*, [s.d.]. Disponível em: <https://tinyurl.com/mrpsr4zm>. Acesso em: 18 dez. 2025.

PAULA FILHO, W. de P. *Engenharia de software: fundamentos, métodos e padrões*. 3. ed. Rio de Janeiro: LTC, 2015.

PAULK, M. C. et al. *The Capability Maturity Model for Software v1.1*. Pittsburgh: SEI – Software Engineering Institute Customer Relations, 1993.

PFLEEGER, S. L. *Engenharia de software*. 2. ed. São Paulo: Prentice Hall, 2004.

PMBOK®. *Um guia do conhecimento em gerenciamento de projetos: guia PMBOK®*. 6. ed. Atlanta: Project Management Institute – PMI®, 2017.

PMBOK®. *Um guia do conhecimento em gerenciamento de projetos: guia PMBOK®*. 7. ed. Atlanta: Project Management Institute – PMI®, 2021.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 7. ed. Rio de Janeiro: McGraw-Hill, 2011.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Rio de Janeiro: McGraw-Hill, 2016.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 9. ed. Rio de Janeiro: McGraw-Hill, 2021.

PROGRAMA de gerenciamento de riscos do Microsoft 365. *Microsoft Ignite*, 29 set. 2025. Disponível em: <https://tinyurl.com/4u3ked8z>. Acesso em: 16 dez. 2025.

REZENDE, D. A. *Engenharia de software e sistemas de informação*. 3. ed. Rio de Janeiro: Brasport, 2005.

Rocha, R. S. *Modelagem do processo de desenvolvimento e manutenção de software para a UINFOR/UESB*. Vitória da Conquista: Uinfor/UESB, 2006.

RUSTAMOV, A. Simplified and detailed big picture of scaled agile framework, SAgile®. *LinkedIn*, 2019. Disponível em: <https://tinyurl.com/ypjrzx33>. Acesso em: 30 dez. 2025.

SLACK, N. *Administração da produção*. São Paulo: Atlas, 2006.

SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SOMMERVILLE, I. *Engenharia de software*. 10. ed. São Paulo: Pearson Prentice Hall, 2016.

SOMMERVILLE, I. *Engenharia de software*. 11. ed. São Paulo: Pearson Prentice Hall, 2021.

STAIR, R. M.; REYNOLDS, G. W. *Princípios de sistemas de informação: uma abordagem gerencial*. 6. ed. São Paulo: Pioneira Thomson Learning, 2006.

SOUZA, C. S. *et al. Projeto de interfaces de usuário: perspectivas cognitivas e semióticas*. Rio de Janeiro: PUC, 2000.

SUSNJARA, S.; SMALLEY, I. O que é teste de software? IBM, [s.d.]. Disponível em: <https://tinyurl.com/4vbzzusj>. Acesso em: 20 jan. 2026.

TANENBAUM, A. S. *Organização estruturada de computadores*. São Paulo: Pearson Prentice Hall, 2013.

TRIBUNAL DE CONTAS DA UNIÃO (TCU). *Manual de medição funcional de software: Versão 4.0*. Distrito Federal, 2016.

WASHIZAKI, H. *SWEBOK® – Guide to the Software Engineering Body of Knowledge v4.0a*. Tóquio: Waseda University, 2025.

VARGAS, R. *Manual prático do plano de projeto: utilizando o PMBOK® guide*. 4. ed. Rio de Janeiro: Brasport, 2011.

VERSOLATTO, F. R. *Projeto de sistemas orientado a objetos*. São Paulo: Editora Sol, 2015.

VASQUES, C. E.; SIMÕES, G. S.; ALBERT, R. M. *Análise de pontos de função*. São Paulo: Érica, 2016.

VIEIRA, E. L.; WAZLAWICK, R. S. *Teoria explanatória para estimativa baseada em casos de uso no desenvolvimento orientado a objetos*. Santa Catarina: Universidade Federal de Santa Catarina – UFSC, 2006.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



Handwriting practice lines consisting of 30 horizontal blue lines. Each line is preceded by a small blue dot, serving as a guide for letter height and placement.



Informações:
www.sepi.unip.br ou 0800 010 9000